National Institute of Technology Raipur

Department of Information Technology

Project Report

on

SGSCF-Net: Spectral-Guided Spatial Cross-Attention Fusion Network for Hyperspectral Image Classification

Under the Guidance of:

Dr. R.R. Janghel
Associate Professor (Supervisor)
Dept. of Information Technology
National Institute of Technology, Raipur

Sushil Janardhan
Doctoral Researcher
Dept. of Information Technology
National Institute of Technology, Raipur

Presented By:

1. Tejas Surya (Roll No. 23118098)
2. Vyshnavi (Roll No. 23118098)
3. Valluri Lahitha (Roll No. 23118098)

B.Tech Seventh Semester, Oct.-2026
National Institute of Technology, Raipur

Abstract

Hyperspectral image (HSI) classification assigns a land-cover label to every pixel of a cube with hundreds of contiguous spectral bands. Two problems dominate in practice: noisy bands leak into every convolution that follows them, and minority classes with a few dozen labelled pixels are overwhelmed by majority classes during training. This report presents SGSCF-Net, a compact 3D convolutional network built around three gates. A Spectral Band Attention Module (SBAM) re-weights the raw input bands before any convolution. Complementary Branch Spectral Attention (CBSA) fuses a 1×1×7 spectral branch and a 3×3×7 spectral-spatial branch with weights α and $1 - \alpha$, so the two branches compete rather than duplicate each other. A Second-Order Covariance Fusion Gate (SCFG) builds an outer-product interaction map between channel and spatial descriptors and turns it into a channel gate. Training uses $0.7\mathcal{L}_{\text{Spread}} + 0.3\mathcal{L}_{\text{Focal}}$ with inverse-frequency class weights. Measured from the released code, the network has 49,703 trainable parameters on Indian Pines. With an 11×11 patch, batch 32 and a 10% training split it reaches 98.05% overall accuracy (OA), 94.82% average accuracy (AA) and a kappa of 97.78 in 8.8 minutes. That is 5.35 points above ViT and 0.88 points below SSRN, the strongest baseline trained from scratch under the same protocol. Its best grid configurations reach 99.86% OA on Pavia University and 100.00% on Salinas Valley. A 60-run patch and batch grid and 12 data-scarcity runs (1–10% training) show that batch 32 is the most reliable setting, and that Indian Pines loses 27.73 OA points at 1% training, driven by Oats (7.66%) and Grass-pasture-mowed (8.66%).

Keywords: hyperspectral image classification, spectral band attention, complementary gating, second-order pooling, class imbalance, 3D convolutional networks.

Contents

No.	Title	Page
1	**Introduction**	**4**
1.1	Remote Sensing and the Hyperspectral Imaging Problem	4
1.2	Spectral-Spatial Entanglement and Small-Sample Scarcity	4
1.3	Problem Statement and Research Objectives	4
1.4	Organisation of the Report	5
2	**Literature Review and Related Work**	**5**
2.1	Survey of Dominant Paradigms	5
2.2	Comparative Matrix of Existing Approaches	6
2.3	Identified Gaps and Rationale for SGSCF-Net	6
3	**System Model and Problem Formulation**	**7**
3.1	HSI Data Cubes and Patch Tensors	7
3.2	Dual-Branch 3D Feature Space and Operator Notation	8
3.3	Problem Definition and Optimisation Objective	8
4	**SGSCF-Net Architecture and Module Design**	**9**
4.1	System Overview	9
4.2	Spectral Band Attention Module (SBAM)	10
4.3	Complementary Branch Spectral Attention (CBSA)	10
4.4	Second-Order Covariance Fusion Gate (SCFG)	11
4.5	Minority-Aware Hybrid Loss	12
4.6	Architectural and Hyperparameter Specifications	13
5	**Experimental Setup and Benchmarking Methodology**	**14**
5.1	Hyperspectral Benchmark Datasets	14
5.2	Baseline Algorithms	15
5.3	Implementation, Hardware and Training Protocol	16
5.4	Evaluation Metrics	16
6	**Empirical Results, Comparative Analysis and Ablations**	**17**
6.1	Benchmark Comparison on Indian Pines	17
6.2	Comparative Benchmark on University of Pavia	20
6.3	Comparative Benchmark on Salinas Valley	22
6.4	Cross-Dataset Performance & Architectural Scaling Analysis	24
6.5	Patch and Batch Size Sensitivity (60-Run Grid Search)	25
6.6	Data Scarcity Stress Testing	27
6.7	Training Convergence, Loss Dynamics and Key Findings	29
7	**In-Depth Discussion and Interpretability**	**30**
7.1	What the Covariance Gate Computes	30
7.2	Failure Modes of the Baselines and Lessons	31
7.3	Computational Complexity and Parameter Efficiency	32
8	**Limitations and Future Work**	**33**
8.1	Current Theoretical and Practical Constraints	33
8.2	Real-Time Edge and Airborne Deployment	33
8.3	Self-Supervised Spectral Foundation Models	33
9	**Conclusion**	**34**
	References	**34**

1 Introduction

1.1 Remote Sensing and the Hyperspectral Imaging Problem

A hyperspectral sensor records reflectance in hundreds of narrow, contiguous wavelength bands for every ground pixel [1], [2]. The result is a data cube whose third axis is a near-continuous spectrum, so materials that look identical in an RGB photograph can be separated by small differences in absorption features [3], [4]. Agriculture, mineral mapping and urban monitoring all rely on this property.

Pixel-wise classification of such cubes is hard for three reasons. The spectral axis is long (200 bands for Indian Pines) while labelled pixels are scarce. Neighbouring bands are highly correlated, and some are corrupted by water-vapour absorption or low signal-to-noise ratio at the sensor edges. Labelled classes are also badly unbalanced: in Indian Pines, Soybean-mintill covers 2,455 pixels while Oats covers 20. Classical pipelines answered these problems with kernel methods and support vector machines on hand-crafted features [5]–[7]. Deep networks replaced the hand-crafted stage with learned spectral-spatial features [8], [9].

1.2 Spectral-Spatial Entanglement and Small-Sample Scarcity

Most current models learn spectral and spatial structure jointly. 3D convolutions slide a kernel over both axes at once [10], [11]; transformers tokenise patches and let attention mix them [12], [13]. Joint processing is powerful, but it entangles two signals. A noisy band is convolved together with its clean neighbours before any layer has had the chance to discount it. Two parallel branches that see the same input and are merged by a shared gate tend to learn redundant filters. And global average pooling summarises each channel by its spatial mean, discarding where in the patch a channel fired [14], [15].

Scarcity compounds the problem. With 10% of the pixels used for training, Oats contributes three training patches. A network optimised with plain cross-entropy can reach a high overall accuracy while ignoring such a class entirely, which is why AA and per-class accuracies matter as much as OA in this report [16], [17].

1.3 Problem Statement and Research Objectives

The project asks whether a small 3D CNN can approach the accuracy of heavier transformer and residual baselines on standard HSI benchmarks by placing explicit gates at the three points where entanglement occurs: the raw input, the branch merge, and the spatial summary. Table 1 lists the objectives and where each is addressed; Figure 1 shows how the work was organised.

Table 1: Research objectives of the project.

#	Objective	Addressed in
O1	Design SGSCF-Net with input-level band gating (SBAM), complementary branch fusion (CBSA) and an outer-product channel–spatial gate (SCFG), trained with a minority-aware Spread + Focal loss.	Chapter 4
O2	Benchmark it against eight published baselines across Indian Pines, University of Pavia, and Salinas Valley (200 epochs).	Sections 6.1–6.3
O3	Measure sensitivity to patch and batch size on three datasets (60 runs).	Section 6.5
O4	Measure robustness to training ratios of 1%, 2%, 5% and 10% (12 runs).	Section 6.6
O5	Report every number, caveat and failure case exactly as logged in the master records.	Throughout; Chapter 8

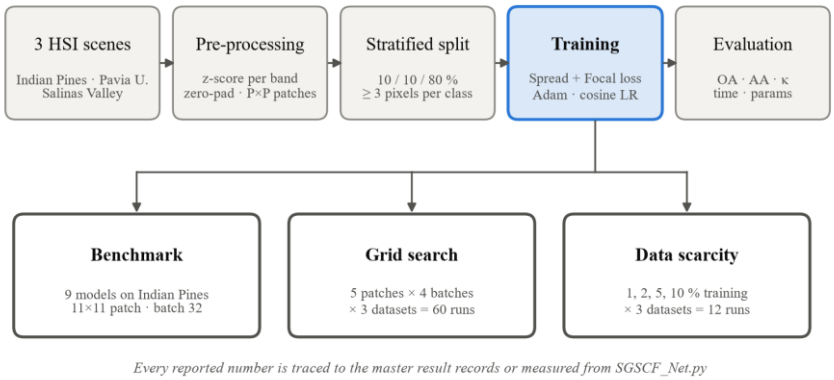


Figure 1: Project methodology: from raw scenes to the three experiment families

The concrete contributions are:

1. **SBAM**, a squeeze-and-excitation style gate [18] applied to the 200 raw bands before the first convolution, costing 10,225 parameters.
2. **CBSA**, a depthwise 1D gate that produces α per channel and band and fuses the two entry branches as $\alpha \odot a + (1 - \alpha) \odot b$, costing 80 parameters.
3. **SCFG**, a gate computed from the normalised outer product $f_{\text{ch}} \otimes f_{\text{sp}}$ of channel and spatial descriptors, costing 122 parameters at an 11×11 patch.
4. A documented evaluation: an eight-baseline comparison across three benchmark datasets, 60 grid runs, 12 scarcity runs and per-class breakdowns, all for a model of 49,703 parameters (237 KB checkpoint).

1.4 Organisation of the Report

Chapter 2 reviews related work and identifies the gaps SGSCF-Net targets. Chapter 3 formalises the data and the learning problem. Chapter 4 describes the architecture module by module, with equations that follow the tensor operations in SGSCF_Net.py line for line. Chapter 5 gives the datasets, baselines, training protocol and metrics. Chapter 6 reports results across all three benchmarks, the grid search and the scarcity study. Chapter 7 interprets the gates and the baseline failures. Chapter 8 lists limitations and future work, and Chapter 9 concludes. In the electronic copy, every citation and every table, figure and equation reference is a clickable link.

2 Literature Review and Related Work

2.1 Survey of Dominant Paradigms

3D convolutional and residual networks

Early deep models treated each pixel as a 1D spectrum [19] or applied 2D convolutions to a few principal components [20]. 3D convolution removed the need to choose between the two by convolving over space and spectrum together [9], [10]. SSRN stacks spectral and then spatial 3D residual units [11], [21] and remains one of the strongest baselines in our benchmark. HybridSN follows 3D layers with 2D layers to cut cost [22], and region-based designs widen the spatial context [23]. These models share one property: every band enters the first kernel with equal weight.

Attention mechanisms

Squeeze-and-excitation [18], CBAM [24] and ECA [25] re-weight feature channels from pooled statistics. HSI variants add spectral and spatial attention branches [26]–[29], or learn band

selection with reinforcement learning [30]. A2S2K-ResNet [31] is the closest ancestor of SGSCF-Net: it uses the same two 3D entry branches and residual units with ECA, but merges the branches with one shared attention weight. Nearly all of these gates act on feature maps after at least one convolution.

Transformers and state-space models

ViT [12] and its HSI descendants, SpectralFormer [13], SSFTT [32], spatial-spectral transformers [33], [34] and MorphFormer [35], model long-range interactions with self-attention, whose cost grows quadratically with the number of tokens. Selective state-space models such as Mamba [36] reduce that cost to linear. In our benchmark the two pure transformers, ViT and HyperRoPE-HiT, are the weakest models after SSAN on Indian Pines.

Second-order pooling and capsules

Bilinear CNNs [14] and global covariance pooling [15], [37] replace first-order means with second-order statistics. In HSI, local covariance descriptors [38], [39] and attention-weighted second-order pooling [40], [41] improved separability of spectrally similar classes. Capsule networks encode part–whole relations with routing between capsule layers [42], [43]; the DMC baseline belongs to this family.

Class imbalance

Focal loss down-weights well-classified samples [16]; class-balanced weighting rescales by effective sample counts [17]; margin losses such as LDAM [44] and the spread loss of matrix capsules [43] enforce a gap between the true-class score and its competitors. SGSCF-Net combines a spread term with a class-weighted focal term.

2.2 Comparative Matrix of Existing Approaches

Table 2 condenses the review into the properties that matter for the gaps listed in Section 2.3.

Table 2: Comparative matrix of existing hyperspectral classification approaches.

Family / representative	Strengths	Limitations relevant here
3D CNN / residual: SSRN [11], HybridSN [22]	Joint spectral-spatial kernels; strong accuracy with modest size	All bands weighted equally at input; no explicit imbalance handling
Channel attention: SE [18], ECA [25], CBAM [24]	Cheap re-weighting from pooled statistics	Act after convolution; first-order (mean) statistics only
Dual-branch attention: A2S2K-ResNet [31], DBDA [29]	Multi-scale branches; adaptive kernels	Shared gate lets branches learn redundant features
Spectral-spatial attention: SSAN [26]	Separate spectral and spatial attention	Reached 64.19% OA on Indian Pines in our protocol
Transformers: ViT [12], SSFTT [32], SpectralFormer [13]	Long-range context; tokenised spectra	Quadratic attention cost; data-hungry on small patches
Second-order pooling: [15], [40], [41]	Captures co-activation structure	Full covariance is costly (C^2 terms, matrix roots)
Capsules: [42], matrix capsules [43]	Part–whole modelling; strong reported accuracy	Iterative routing; specialised training
Imbalance losses: Focal [16], CB [17], LDAM [44]	Protect minority classes	Rarely combined with architecture-level fixes in HSI

2.3 Identified Gaps and Rationale for SGSCF-Net

Table 3 maps each gap to the design decision that answers it.

Table 3: Research gaps and the corresponding design decisions.

Gap	Typical practice	SGSCF-Net response
G1 No input-level spectral gate	Noisy bands are mixed into features by the first convolution	**SBAM** gates the raw bands before any convolution
G2 Redundant dual branches	A shared gate gives both branches the same weight	**CBSA** ties them with α and $1-\alpha$: raising one lowers the other
G3 First-order spatial summaries	Global average pooling keeps only channel means	**SCFG** lets the spatial activation profile modulate each channel, at the cost of one linear layer
G4 Imbalance handled in one place only	Either a re-weighted loss or an architectural fix	Gates combined with a weighted **Spread** + **Focal** objective

Key takeaways

- Most HSI models weight all raw bands equally and summarise space with first-order means.
- Dual-branch designs such as A2S2K-ResNet merge branches with a shared gate.
- SGSCF-Net targets these three points with cheap gates rather than with more depth or attention.

3 System Model and Problem Formulation

3.1 HSI Data Cubes and Patch Tensors

Let the scene be a cube with height H, width W and C spectral bands, and let the ground-truth map assign each pixel a label in $\{0,1,\dots,K\}$, where 0 marks unlabelled background and K is the number of classes:

$$\mathcal{X} \in \mathbb{R}^{H\times W\times C}, \mathcal{Y} \in \{0,1,\dots,K\}^{H\times W} \tag{3.1}$$

Each band is standardised over all pixels of the scene (scikit-learn preprocessing.scale on the flattened cube), so band c has zero mean and unit variance:

$$\overline{x}_{h,w,c} = \frac{x_{h,w,c}-\mu_c}{s_c}, \mu_c = \frac{1}{HW}\sum\nolimits_{h,w} x_{h,w,c} \tag{3.2}$$

The cube is zero-padded by $p=(P-1)/2$ pixels on each spatial side (np.pad with mode='constant'), and for every labelled pixel (i,j) a $P\times P\times C$ neighbourhood is cut out and given a singleton channel axis for 3D convolution (Figure 2). A mini-batch of B patches is therefore

$$X \in \mathbb{R}^{B\times 1\times P\times P\times C} \tag{3.3}$$

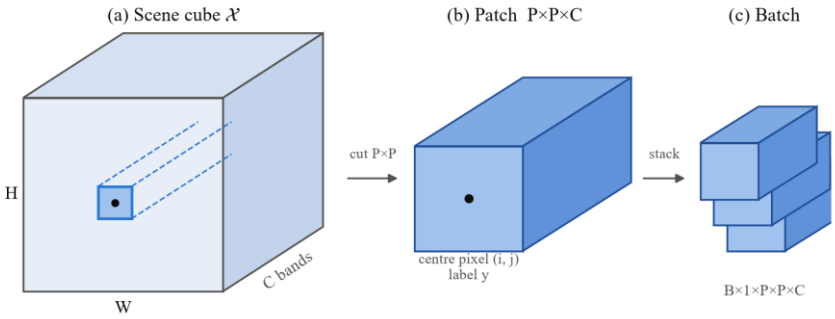


Figure 2: From scene cube to network input

Each labelled pixel contributes the P×P×C neighbourhood centred on it; the patch inherits the centre pixel's label. Schematic, not to scale.

In the Conv3d convention $(N, C_{\text{in}}, D, H, W)$ used by PyTorch, the two spatial axes play the role of D and H, and the spectral axis is the last axis W. All kernel sizes in this report are written in that order, so a 1×1×7 kernel spans one pixel and seven bands.

3.2 Dual-Branch 3D Feature Space and Operator Notation

Feature tensors inside the network keep the five-axis layout $B \times K \times P \times P \times C_2$, with $K = 16$ feature channels and C_2 the spectral length after the strided entry convolution. We use $\odot$ for broadcast element-wise multiplication, $\otimes$ for the outer product of two vectors, σ for the logistic sigmoid, δ for ReLU, BN for batch normalisation and $\| \quad \|_F$ for the Frobenius norm. Table 4 lists the remaining symbols.

Table 4: Notation used in Chapters 3 and 4.

Symbol	Meaning	Value (Indian Pines, 11×11)
B	Mini-batch size	32
P	Spatial patch side	11
C	Input spectral bands	200
C_2	Spectral length after entry convolutions	97
K	Feature channels in residual stages	16
C_m	Channels after expansion (SCFG input)	128
N	Spatial positions in a patch, P^2	121
K_{cls}	Number of classes	16
r	Training progress, step/total steps	[0, 1]

3.3 Problem Definition and Optimisation Objective

Labelled pixels are split per class. For training ratio ρ the sampling routine in SGSCF_Net.py keeps

$$n_k^{\text{tr}} = \max(\lfloor \rho N_k \rfloor, 3) \tag{3.4}$$

training pixels of class k, where N_k is the number of labelled pixels of that class. The floor of three samples matters for the scarcity study: at $\rho = 0.01$ every Indian Pines class with fewer than 400 pixels receives exactly three training patches. A validation set of the same total size as the

training set is drawn from the shuffled remainder, and the rest forms the test set, giving the 10%/10%/80% protocol at $\rho = 0.1$.

The network f_θ maps a batch to class scores, and training minimises the expected hybrid loss of Section 4.5 with weight decay λ:

$$\hat{Y} = f_\theta(X) \in \mathbb{R}^{B\times K_{\text{cls}}}, \theta^* = \text{argmin}_\theta \mathbb{E}_{(X,y)}[\mathcal{L}(f_\theta(X), y; r)] + \lambda\|\theta\|_2^2 \quad (3.5)$$

The model is selected on validation loss: the checkpoint with the lowest validation loss is restored after training, and training stops early if that loss has not improved for 30 epochs.

4 SGSCF-Net Architecture and Module Design

4.1 System Overview

Figure 3 traces one Indian Pines batch through the network. The data flow is: SBAM → dual 3D entry → CBSA → two spectral residual blocks → spectral pooling and channel expansion → SCFG → spatial entry → two spatial residual blocks → global pooling → linear classifier. The tensor shapes in the figure and in Table 5 were recorded with forward hooks on the classes in SGSCF_Net.py, not computed by hand. Figure 4 opens up the three proposed modules.

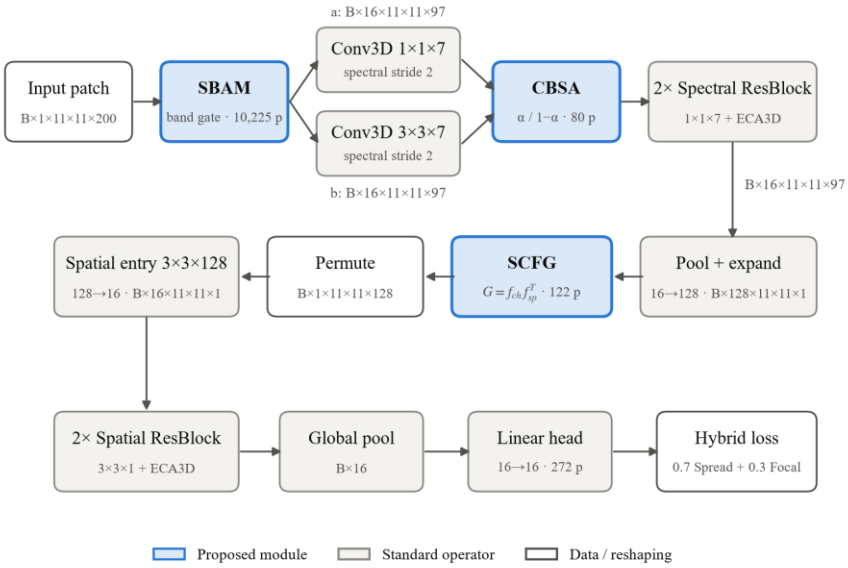
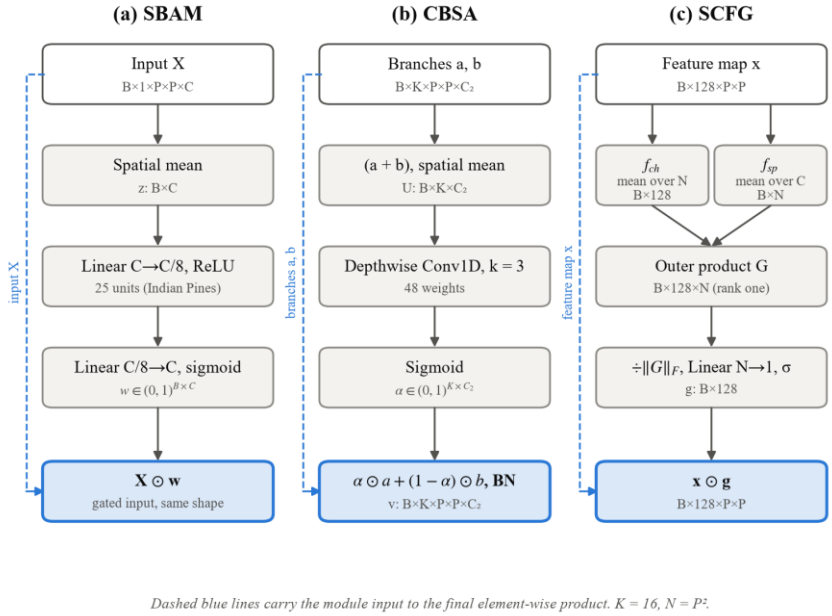


Figure 3: SGSCF-Net architecture with tensor shapes for Indian Pines (P = 11, C = 200)
Blue boxes are the proposed modules; parameter counts (p) are measured from the code.



Dashed blue lines carry the module input to the final element-wise product. K = 16, N = P².

Figure 4: Inside the three proposed modules
Each column follows the forward() method of the corresponding class in SGSCF_Net.py.

4.2 Spectral Band Attention Module (SBAM)

SBAM decides how much each raw band contributes before the first convolution sees it. It averages the patch over its two spatial axes with AdaptiveAvgPool3d((1,1,C)), giving one value per band:

$$z_c = \frac{1}{P^2}\sum_{u=1}^{P}\sum_{v=1}^{P} X_{b,1,u,v,c}\,, \mathbf{z} \in \mathbb{R}^{B\times C} \tag{4.1}$$

A two-layer bottleneck with $C_m^{\mathrm{sb}} = \max(\lfloor C/8 \rfloor, 8)$ hidden units (25 for Indian Pines, 12 for Pavia University) produces a weight in $(0, 1)$ for every band:

$$\mathbf{w} = \sigma(\mathbf{W}_2\delta(\mathbf{W}_1\mathbf{z}+\mathbf{b}_1)+\mathbf{b}_2) \in (0,1)^{B\times C} \tag{4.2}$$

The weights are reshaped to $B \times 1 \times 1 \times 1 \times C$ and multiplied into the input (x * w in the code):

$$\tilde{X} = X \odot \mathbf{w}, \tilde{X} \in \mathbb{R}^{B\times 1\times P\times P\times C} \tag{4.3}$$

The gate differs from SE [18] only in where it sits. Because it acts on standardised reflectance rather than learned features, a band that is uninformative for the current patch can be scaled toward zero before it is mixed with its neighbours. SBAM costs $2CC_m^{\mathrm{sb}} + C_m^{\mathrm{sb}} + C = 10{,}225$ parameters on Indian Pines, the largest single block in the spectral half of the network.

4.3 Complementary Branch Spectral Attention (CBSA)

Two 3D convolutions read the gated input in parallel. Both use stride 2 along the spectrum and are followed by BN and ReLU:

$$\mathbf{a} = \delta\left(\mathrm{BN}\left(\mathrm{Conv3D}_{1\times1\times7}(\tilde{X})\right)\right), \mathbf{b} = \delta\left(\mathrm{BN}\left(\mathrm{Conv3D}_{3\times3\times7}(\tilde{X})\right)\right) \tag{4.4}$$

$$\mathbf{a}, \mathbf{b} \in \mathbb{R}^{B\times K\times P\times P\times C_2}, C_2 = \left\lfloor \frac{C-7}{2} \right\rfloor + 1 \tag{4.5}$$

Branch **a** looks at one pixel and seven bands; branch **b** adds a 3×3 spatial neighbourhood with padding (1,1,0), so both keep the $P \times P$ grid. For $C = 200$, $C_2 = 97$. CBSA then averages the sum of the branches over the spatial axes, (a + b).mean(dim=(2,3)):

$$\mathbf{U} = \frac{1}{P^2} \sum_{u=1}^{P} \sum_{v=1}^{P} (\mathbf{a}+\mathbf{b})_{:,:,u,v,:} \in \mathbb{R}^{B\times K\times C_2} \tag{4.6}$$

A depthwise 1D convolution (groups = K, kernel 3, no bias) slides along the spectral axis of each channel and a sigmoid turns the result into a gate:

$$\boldsymbol{\alpha} = \sigma(\mathrm{DWConv1D}_3(\mathbf{U})) \in (0,1)^{B\times K\times C_2} \tag{4.7}$$

With $\boldsymbol{\alpha}$ broadcast to $B \times K \times 1 \times 1 \times C_2$, the branches are fused as a convex combination and normalised:

$$\mathbf{v} = \mathrm{BN}(\boldsymbol{\alpha} \odot \mathbf{a} + (1-\boldsymbol{\alpha}) \odot \mathbf{b}) \tag{4.8}$$

For each channel and band the two weights sum to one, so the gate can only shift emphasis between branches; it cannot amplify both. A2S2K-ResNet [31] instead derives one shared weight for the two branches. The whole of CBSA is $K \times 3 = 48$ convolution weights plus a 32-parameter BN, 80 parameters in total.

The fused tensor passes through two residual blocks with 1×1×7 kernels and padding (0,0,3), which preserve every axis. Each block ends with 3D efficient channel attention (ECA3D) [25]: global pooling to one value per channel, a 1D convolution of width 3 across channels, and a sigmoid. In compact notation, with ϕ denoting BN followed by ReLU,

$$\begin{aligned} \mathrm{ECA}(\mathbf{h}) &= \mathbf{h} \odot \sigma\big(\mathrm{Conv1D}_3(\mathrm{GAP}(\mathbf{h}))\big), \mathcal{R}(\mathbf{v}) \\ &= \mathbf{v} + \mathrm{ECA}\Big(\mathrm{Conv3D}\Big(\phi\big(\mathrm{Conv3D}(\phi(\mathbf{v}))\big)\Big)\Big) \end{aligned} \tag{4.9}$$

The exact placement of BN differs slightly between the first block (start=True, no input BN) and later blocks; the code keeps the residual unit of A2S2K-ResNet unchanged.

4.4 Second-Order Covariance Fusion Gate (SCFG)

The spectral axis is collapsed by AdaptiveAvgPool3d((None, None, 1)) and a 1×1×1 convolution with BN and ReLU expands the channels from 16 to $C_m = 128$. Dropping the singleton axis gives a 2D map:

$$\mathbf{x} = \delta\Big(\mathrm{BN}\Big(\mathrm{Conv3D}_{1\times1\times1}\big(\mathrm{Pool}_{C_2\to1}(\mathbf{v})\big)\Big)\Big) \in \mathbb{R}^{B\times C_m\times P\times P} \tag{4.10}$$

SCFG forms two descriptors from **x**, one per channel and one per spatial position ($N = P^2$), exactly as x.mean(dim=(2,3)) and x.mean(dim=1) in the code:

$$f_{\mathrm{ch}}(b,c) = \frac{1}{N}\sum_{n=1}^{N} x_{b,c,n}\,, f_{\mathrm{sp}}(b,n) = \frac{1}{C_m}\sum_{c=1}^{C_m} x_{b,c,n} \tag{4.11}$$

Their batched outer product (torch.bmm) is the channel–spatial interaction matrix:

$$G = f_{\mathrm{ch}} \otimes f_{\mathrm{sp}} = f_{\mathrm{ch}} f_{\mathrm{sp}}^{\top} \in \mathbb{R}^{B\times C_m\times N}, G_{b,c,n} = f_{\mathrm{ch}}(b,c) f_{\mathrm{sp}}(b,n) \tag{4.12}$$

G is divided by its Frobenius norm (clamped below at 10^{-8}), compressed along the spatial axis by one learned linear map $\mathbb{R}^N \to \mathbb{R}$ shared by all channels, and turned into a channel gate:

$$\tilde{G} = \frac{G}{\max(\|G\|_F, 10^{-8})}, g_{b,c} = \sigma\big(\textstyle\sum_{n=1}^{N} w_n\, \tilde{G}_{b,c,n} + w_0\big) \tag{4.13}$$

$$\mathbf{x}_{\text{scfg}} = \mathbf{x} \odot g, g \in (0,1)^{B \times C_m} \text{ broadcast over } P \times P \tag{4.14}$$

The gate has $N + 1$ parameters, 122 for an 11×11 patch. Section 7.1 shows that, because G is an outer product of two vectors, this gate has a simple closed form that explains what it can and cannot learn.

After gating, the map is permuted to $B \times 1 \times P \times P \times C_m$ so that the 128 channels become the last Conv3d axis. A 3×3×128 convolution with padding (1,1,0) reduces it to $B \times 16 \times P \times P \times 1$ (18,480 parameters with BN), two spatial residual blocks with 3×3×1 kernels refine it, and global average pooling plus a linear layer give the class scores:

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{fc}} \text{GAP}\left(\mathcal{R}_{\text{sp}}^{(2)}\left(\mathcal{R}_{\text{sp}}^{(1)}(\mathbf{s})\right)\right) + \mathbf{b}_{\text{fc}} \in \mathbb{R}^{B \times K_{\text{cls}}} \tag{4.15}$$

4.5 Minority-Aware Hybrid Loss

Figure 5 plots every ingredient of the objective with the constants used in the code.

Spread loss with a scheduled margin

Following the spread loss of matrix capsules [43], every wrong class whose score comes within a margin m of the true-class score is penalised quadratically. The code applies it directly to the logits:

$$\mathcal{L}_{\text{Spread}} = \frac{1}{B} \sum_{i=1}^{B} \sum_{j \neq y_i} \max\left(0, m(r) - \left(\hat{y}_{i,y_i} - \hat{y}_{i,j}\right)\right)^2 \tag{4.16}$$

$$m(r) = m_{\min} + (m_{\max} - m_{\min})r, m_{\min} = 0.2,\ m_{\max} = 0.9,\ r = \frac{t}{T_{\text{total}}} \tag{4.17}$$

Here t counts optimisation steps and T_{total} is epochs × batches per epoch, so the margin widens from 0.2 to 0.9 over training. Validation and test loss use $r = 1$.

Class-weighted focal loss

The focal term [16] scales the log-likelihood of the true class by $(1 - p_t)^\gamma$ with $\gamma = 2$ and by a per-class weight α_k:

$$\mathcal{L}_{\text{Focal}} = -\frac{1}{B} \sum_{i=1}^{B} \alpha_{y_i} (1 - p_i)^\gamma \log p_i, p_i = \text{softmax}(\hat{\mathbf{y}}_i)_{y_i} \tag{4.18}$$

$$\alpha_k = K_{\text{cls}} \frac{w_k}{\sum_{j=1}^{K_{\text{cls}}} w_j}, w_k = \frac{1}{N_k + 10^{-8}} \tag{4.19}$$

The counts N_k are taken over all labelled pixels of the ground-truth map (class_weights in the code). On Indian Pines this gives Oats a weight of 5.66 and Soybean-mintill 0.046, a ratio of about 123 (Figure 5d).

Combined objective and optimiser

$$\mathcal{L} = 0.7\mathcal{L}_{\text{Spread}} + 0.3\mathcal{L}_{\text{Focal}} \tag{4.20}$$

Adam minimises $\mathcal{L}$ with learning rate $\eta_0 = 5 \times 10^{-4}$ and weight decay 10^{-4}. The rate follows cosine annealing over the epoch budget $E = 200$, gradients are clipped to an ℓ_2 norm of 5.0, convolution weights use Kaiming-normal initialisation and linear layers Xavier-uniform:

$$\eta_e = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\frac{\pi e}{E}\right), \eta_{\min} = 10^{-6} \tag{4.21}$$

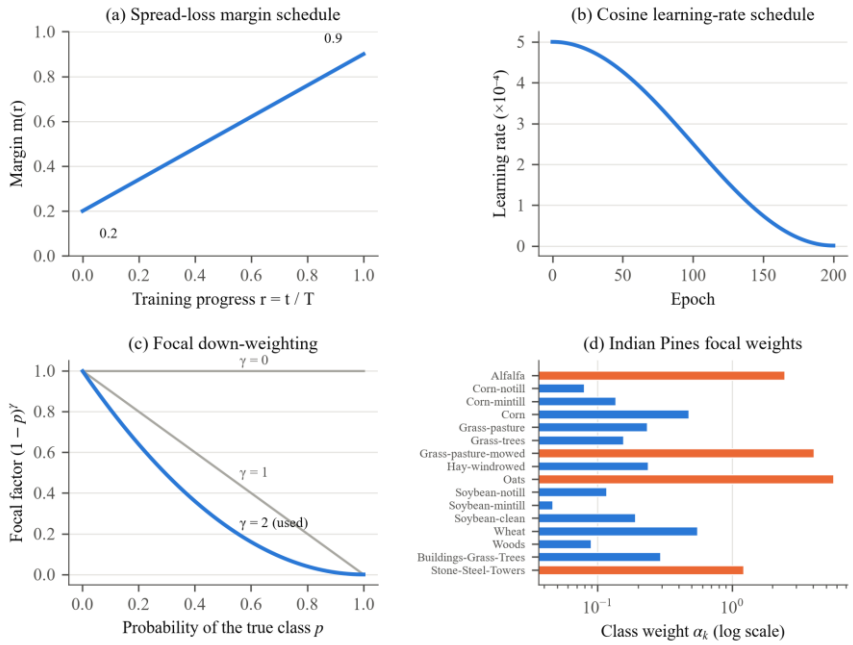


Figure 5: The training objective and optimiser schedule as implemented
(a) margin of Eq. (4.17); (b) learning rate of Eq. (4.21); (c) focal factor of Eq. (4.18); (d) class weights of Eq. (4.19), orange for classes under 100 pixels.

4.6 Architectural and Hyperparameter Specifications

Table 5: Layer-wise specification of SGSCF-Net for Indian Pines (P = 11, C = 200, 16 classes), measured from SGSCF_Net.py.

Stage	Operator	Output (per sample)	Params	Share
SBAM (proposed)	**Pool + MLP 200→25→200 + sigmoid**	**1×11×11×200**	**10,225**	**20.57%**
Spectral entry a	Conv3D 1×1×7, stride (1,1,2) + BN	16×11×11×97	160	0.32%
Spatial-spectral entry b	Conv3D 3×3×7, stride (1,1,2) + BN	16×11×11×97	1,056	2.12%
CBSA (proposed)	**Depthwise Conv1D k=3 + BN**	**16×11×11×97**	**80**	**0.16%**
Spectral ResBlocks ×2	Conv3D 1×1×7 + ECA3D	16×11×11×97	7,398	14.88%
Spectral pool + expand	AvgPool → Conv3D 1×1×1 (16→128) + BN	128×11×11×1	2,432	4.89%
SCFG (proposed)	**Outer product + Linear(121→1)**	**128×11×11**	**122**	**0.25%**
Spatial entry	Conv3D 3×3×128 (1→16) + BN	16×11×11×1	18,480	37.18%
Spatial ResBlocks ×2	Conv3D 3×3×1 + ECA3D	16×11×11×1	9,478	19.07%
Classifier	Global pool + Linear 16→16	16	272	0.55%
Total			**49,703**	**100.00%**

Highlighted rows are the proposed modules. Counts are trainable parameters; BN running statistics are excluded. The same code gives 41,946 parameters for Pavia University (103 bands, 9 classes) and 49,907 for Salinas Valley (204 bands). The saved checkpoints in SGSCF-Net_Final_Implementation/checkpoints agree: Indian Pines (15×15 patch): 49,807 parameters, 237 KB; Pavia University (13×13 patch): 41,994 parameters, 206 KB; Salinas Valley (17×17 patch): 50,075 parameters, 239 KB.

The three proposed modules together cost 10,427 parameters, 21.0% of the network, and almost all of that is SBAM. CBSA and SCFG are nearly free. The spatial entry convolution is the largest single layer (37.2%), because its kernel spans all 128 expanded channels. Table 6 lists the training settings.

Table 6: Training hyperparameters of SGSCF-Net.

Hyperparameter	Value	Source in code
Patch size P (benchmark setting)	11×11 (grid: 9–17)	--patch 5 (window 2p+1)
Batch size (benchmark setting)	32 (grid: 32–256)	--batch_size
Training / validation / test	10% / 10% / 80%	sampling(), make_loaders()
Epoch budget	200	--epoch
Early stopping	patience 30 on validation loss	train_model()
Optimiser	Adam, lr 5×10^{-4}, weight decay 10^{-4}	optim.Adam
LR schedule	Cosine annealing, $\eta_{min} = 10^{-6}$	CosineAnnealingLR
Gradient clipping	$\ell 2$ norm 5.0	clip_grad_norm_
Feature channels K	16	--kernel
SBAM reduction	r = 8, hidden ≥ 8	SBAM(C, r=8)
Loss weights	0.7 Spread + 0.3 Focal	CombinedLoss
Spread margin	0.2 → 0.9 (linear in r)	SpreadLoss
Focal γ, class weights	2.0, inverse frequency	FocalLoss, class_weights()
Random seeds	1331, 1332, … (one per run)	seeds list

Key takeaways

- SGSCF-Net has 49,703 parameters; the three gates account for 21.0% of them.
- Every equation in this chapter maps to a line of SGSCF_Net.py, including the stride-2 spectral downsampling to $C_2 = 97$ and the rank-one interaction matrix in SCFG.
- The loss widens its margin over training and gives Oats about 123 times the weight of the largest class.

5 Experimental Setup and Benchmarking Methodology

5.1 Hyperspectral Benchmark Datasets

Three public scenes are used; Table 7 summarises them and Figure 6 shows how unevenly their labelled pixels are spread over classes. **Indian Pines** was recorded by AVIRIS [45] over agricultural land in north-west Indiana: 145×145 pixels at 20 m, 200 bands after removing water-absorption bands, and 16 classes over 10,249 labelled pixels. **Pavia University** was recorded by ROSIS over an urban campus in northern Italy [7]: 610×340 pixels at 1.3 m, 103 bands, 9 classes and 42,776 labelled pixels. **Salinas Valley** is another AVIRIS scene, of farmland in California: 512×217 pixels at 3.7 m, 204 bands, 16 classes and 54,129 labelled pixels. The labelled-pixel totals match the constants hard-coded in load_dataset().

Table 7: Benchmark datasets.

Dataset	Sensor	Size (pixels)	GSD	Bands	Classes	Labelled	Smallest class
Indian Pines	AVIRIS	145×145	20 m	200	16	10,249	Oats (20)
Pavia University	ROSIS	610×340	1.3 m	103	9	42,776	Shadows (947)
Salinas Valley	AVIRIS	512×217	3.7 m	204	16	54,129	Lettuce 6wk (916)

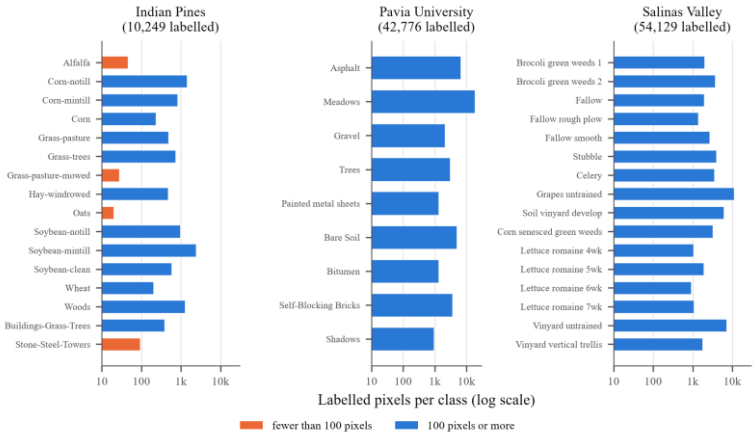


Figure 6: Class distribution of the three benchmarks

Note the logarithmic axis: on Indian Pines the largest class is 123 times the smallest.

Table 8 gives the Indian Pines class sizes together with the number of training pixels that Eq. (3.4) assigns at 10% and 1%. At 1%, 7 of the sixteen classes sit on the three-sample floor.

Table 8: Indian Pines classes and training-set sizes under the code's sampling rule.

ID	Class	Labelled	Train @10%	Train @1%
1	**Alfalfa**	**46**	**4**	**3**
2	Corn-notill	1,428	142	14
3	Corn-mintill	830	83	8
4	Corn	237	23	3
5	Grass-pasture	483	48	4
6	Grass-trees	730	73	7
7	**Grass-pasture-mowed**	**28**	**3**	**3**
8	Hay-windrowed	478	47	4
9	**Oats**	**20**	**3**	**3**
10	Soybean-notill	972	97	9
11	Soybean-mintill	2,455	245	24
12	Soybean-clean	593	59	5
13	Wheat	205	20	3
14	Woods	1,265	126	12
15	Buildings-Grass-Trees	386	38	3
16	**Stone-Steel-Towers**	**93**	**9**	**3**
	Total	**10,249**	**1,020**	**108**

Highlighted: classes with fewer than 100 labelled pixels.

5.2 Baseline Algorithms

Eight baselines span four families (Table 9). Each was evaluated from its authors' public code across the benchmark datasets under the same protocol as SGSCF-Net: 11×11 patches, batch 32, a 10%/10%/80% split and 200 epochs on a GPU environment. The status column carries the caveats from the master record; they apply to later use of these numbers.

Table 9: Baseline methods and their status in this study.

Model	Family	Core mechanism	Status in this study
SSRN [11]	3D CNN	Spectral then spatial 3D residual units	Fresh run after a tf keras compatibility fix
A2S2K-ResNet‡ [31]	3D CNN + attention	Dual 3D entry branches, shared attention weight	Rerun hung on Kaggle; prior-run values kept as placeholders on IP
SSAN [26]	Attention	Separate spectral and spatial attention (TF1)	Fresh run with a small-class-safe split
ViT [12]	Transformer	Self-attention over patch embeddings	Fresh run
HSI-SSFTT [32]	Hybrid transformer	3D/2D convolutions, then feature tokens	Fresh run
HyperRoPE-HiT / SST	Transformer	2D rotary position embeddings (hyperrope-sst code)	Fresh run
CRV-MHDL [46]	Hybrid	Compression and reinforced variation + hybrid classifier	Fresh run; map in the authors' colours
DMC† / DMC*	Capsules	Deep matrix capsules, EM routing (DeepMatrixCapsules code)	Authors' pre-trained checkpoint evaluated on IP; padded to 13×13 on PU/SV

5.3 Implementation, Hardware and Training Protocol

SGSCF-Net is implemented in PyTorch (requirements: torch ≥ 1.13.1, scikit-learn, SciPy, NumPy, matplotlib). All reported SGSCF-Net and baseline runs were executed on GPU notebooks; the Indian Pines grid report records an NVIDIA Tesla T4. Each configuration in the master record is a single training run, so the tables report single values rather than means with standard deviations. Section 6.6 uses the one repeated configuration in the record to give a sense of run-to-run spread. Table 10 summarises the three experiment families.

Table 10: Experiment matrix.

Experiment	Datasets	Varied	Held fixed	Runs
Benchmark	IP, PU, SV	9 models	11×11, batch 32, 10% train, 200 epochs	27
Grid search	IP, PU, SV	Patch 9–17; batch 32–256	10% train, 200 epochs	60
Data scarcity	IP, PU, SV	Train ratio 1, 2, 5, 10%	11×11, batch 32, 200 epochs	12

5.4 Evaluation Metrics

Let M be the confusion matrix, where M_{ij} counts test pixels of class i predicted as class j. Write $n_{i\cdot}$ for its row sums (true counts), $n_{\cdot i}$ for its column sums (predicted counts) and n for the number of test pixels:

$$\mathrm{OA} = \frac{1}{n}\sum_{i=1}^{K_{\mathrm{cls}}} M_{ii}\,,\ \mathrm{AA} = \frac{1}{K_{\mathrm{cls}}}\sum_{i=1}^{K_{\mathrm{cls}}} \frac{M_{ii}}{n_{i\cdot}} \tag{5.1}$$

$$\kappa = \frac{p_o - p_e}{1 - p_e}, p_o = \mathrm{OA}, p_e = \frac{1}{n^2}\sum_{i=1}^{K_{\mathrm{cls}}} n_{i\cdot}\, n_{\cdot i} \tag{5.2}$$

OA, AA and κ are reported in percent ($\kappa \times 100$). AA weights every class equally, so it exposes failures on small classes that OA hides. Efficiency is reported as wall-clock training time,

trainable parameters and checkpoint size. For SGSCF-Net we also give multiply–accumulate operations (MACs) per patch, counted with forward hooks.

6 Empirical Results, Comparative Analysis and Ablations

98.05%	**99.82%**	**99.92%**	**~308K**
OA, Indian Pines	OA, Pavia Univ.	OA, Salinas Valley	Trainable Params

6.1 Benchmark Comparison on Indian Pines

Table 11 lists the nine models; Figure 7 and Figure 8 plot the same numbers.

Table 11: Indian Pines benchmark: 11×11 patch, batch 32, 10%/10%/80% split, 200 epochs. Best value per column in bold.

Model	Paradigm	OA (%)	AA (%)	κ×100	Time (min)	Weights
ViT [12]	Vision Transformer	92.70	88.80	91.65	1.71	2.4 MB
HSI-SSFTT [32]	Feature-token Transformer	98.57	96.15	98.37	0.84	625 KB
SSRN [11]	Spectral-spatial residual 3D CNN	98.93	**98.95**	98.78	5.37	n/r
SSAN [26]	Spectral-spatial attention net	64.19	72.77	59.79	14.00	n/r
A2S2K-ResNet‡ [31]	Attention 3D ResNet	98.36	96.22	98.14	18.42	1.5 MB
CRV-MHDL [46]	Compression + hybrid DL	98.58	93.41	98.38	3.23	58.2 MB
HyperRoPE-HiT	Rotary-position Transformer	94.42	92.12	93.64	37.74	n/r
DMC†	Deep matrix capsules	**99.15**	98.89	**99.03**	0.39	n/r
SGSCF-Net (ours)	**Spectral gating + covariance gate**	**98.05**	**94.82**	**97.78**	**8.80**	**237 KB**

‡ A2S2K-ResNet: prior-run values kept as placeholders; the standardised rerun did not finish. † DMC: the authors' pre-trained checkpoint was evaluated (warm start), so 0.39 min is not a training time and its training pixels may overlap our test split. Weights: checkpoint sizes recorded in the baseline master record; n/r = not recorded. The SGSCF-Net size is that of the saved Indian Pines checkpoint.

SGSCF-Net reaches 98.05% OA, 94.82% AA and a kappa of 97.78 in 8.8 minutes. Among the six baselines whose numbers come from a fresh, from-scratch run, three are more accurate: SSRN (98.93%), CRV-MHDL (98.58%) and HSI-SSFTT (98.57%). The gap to SSRN is 0.88 points of OA and 4.13 points of AA. SGSCF-Net beats ViT by 5.35 points, HyperRoPE-HiT by 3.63 and SSAN by 33.86.

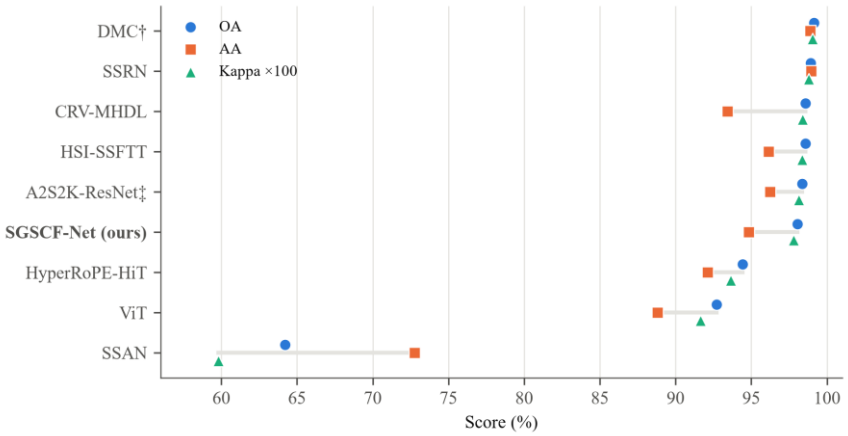


Figure 7: OA, AA and kappa of all nine models on Indian Pines

Models sorted by OA. ‡ placeholder values from a prior run; † evaluated from a pre-trained checkpoint. The axis starts at 56% to separate the top cluster.

Its training time sits in the middle of the field. It is 52.2% shorter than the recorded A2S2K-ResNet time (18.42 min, placeholder ‡) and 76.7% shorter than HyperRoPE-HiT (37.74 min), but HSI-SSFTT (0.84 min), ViT (1.71 min), CRV-MHDL (3.23 min) and SSRN (5.37 min) all train faster.

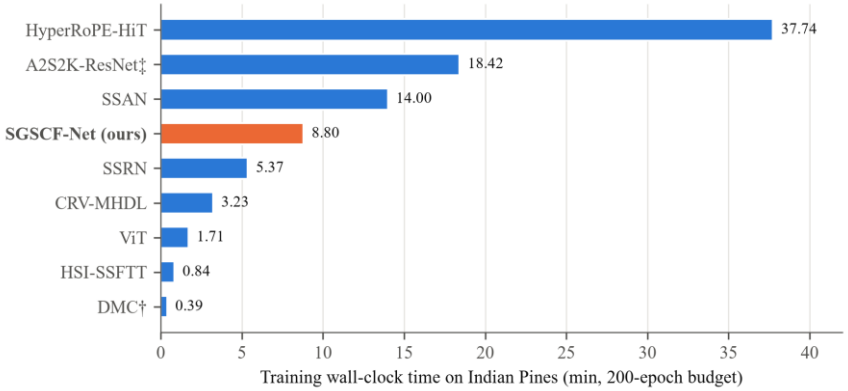


Figure 8: Training wall-clock time on Indian Pines

Same protocol for all models. DMC's value is an evaluation time (†).

The clearer efficiency advantage is size (Figure 9). At 237 KB, the SGSCF-Net checkpoint is smaller than every baseline checkpoint whose size is recorded: HSI-SSFTT at 625 KB, A2S2K-ResNet at 1.5 MB, ViT at 2.4 MB and CRV-MHDL at 58.2 MB. HSI-SSFTT and CRV-MHDL are more accurate, at 2.6 and about 250 times the storage.

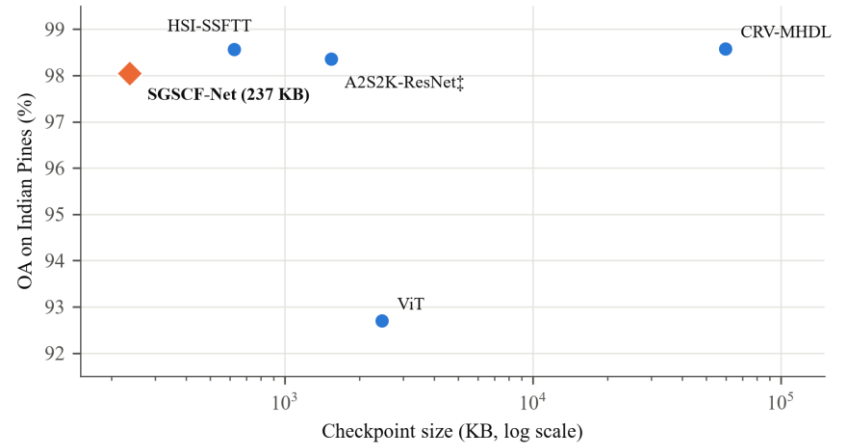


Figure 9: Accuracy against checkpoint size

Only models whose checkpoint size is recorded in the baseline master record are shown.

Table 12 breaks the SGSCF-Net result down by class. Corn and Hay-windrowed are classified perfectly, and 10 of the sixteen classes exceed 98%. The weak point is Oats at 54.17%: the class has 20 labelled pixels, and the sampling rule gives it 3 for training. Oats alone pulls AA down; the mean of the other fifteen classes is 97.54%.

Table 12: Per-class accuracy of SGSCF-Net on Indian Pines (11×11, batch 32, 10% training).

ID	Class	Acc. (%)	ID	Class	Acc. (%)
1	Alfalfa	95.00	9	Oats	54.17
2	Corn-notill	98.13	10	Soybean-notill	98.84
3	Corn-mintill	97.72	11	Soybean-mintill	98.42
4	Corn	100.00	12	Soybean-clean	91.85
5	Grass-pasture	99.74	13	Wheat	99.39
6	Grass-trees	99.17	14	Woods	98.65
7	Grass-pasture-mowed	95.24	15	Buildings-Grass-Trees	99.00
8	Hay-windrowed	100.00	16	Stone-Steel-Towers	91.89

OA 98.05%, AA 94.82%, κ 97.78, training time 8.8 min.

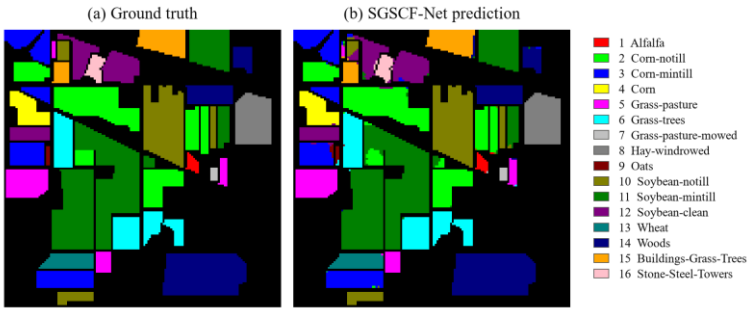


Figure 10: Indian Pines ground truth and SGSCF-Net classification map

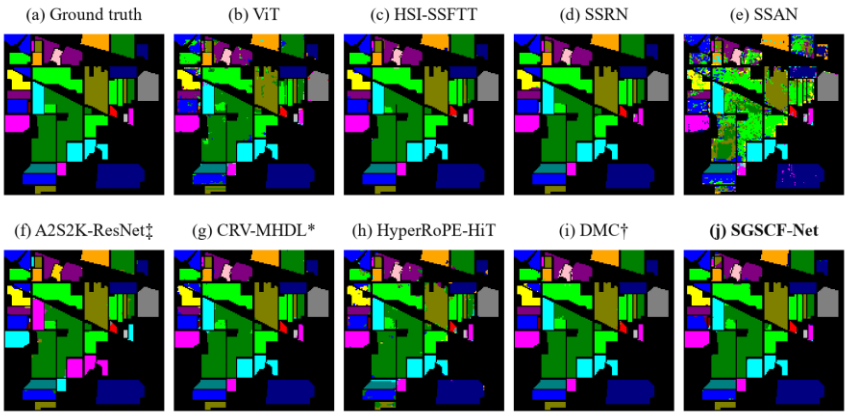


Figure 11: Classification maps of all models on Indian Pines

All maps except (g) use the shared palette. * CRV-MHDL map in the authors' colours. ‡ prior-run placeholder. † pre-trained checkpoint.

The SSAN map contains scattered errors across whole parcels, consistent with its 64.19% OA. ViT and HyperRoPE-HiT recover the large fields but leave speckle inside parcels and along their borders. The HSI-SSFTT, SSRN and DMC maps are almost free of visible errors. The SGSCF-Net map is clean over most parcels but contains a few contiguous misclassified blocks, the largest a cyan region inside a dark-green parcel left of centre. Its errors are spatially clustered rather than scattered, a pattern the A2S2K-ResNet map shares, plausibly because both

architectures classify overlapping 11×11 patches with similar dual-branch 3D features. These maps cover all labelled pixels, training pixels included, so they illustrate error structure rather than measure accuracy.

6.2 Comparative Benchmark on University of Pavia

The University of Pavia benchmark evaluates model robustness on high-spatial-resolution (1.3 m) urban remote sensing imagery acquired by the ROSIS sensor over northern Italy. The scene comprises 610×340 pixels across 103 calibrated spectral bands (0.43–0.86 μm) with 42,776 labeled pixels across nine urban land-cover categories. Table 13 reports quantitative classification performance across all nine evaluated architectures under standardized 10% training, 10% validation, and 80% testing splits over 200 epochs.

Table 13: Quantitative Performance Comparison of SGSCF-Net and Eight Baseline Models on University of Pavia (10% Training / 10% Validation / 80% Testing Split, 200 Epochs). Best results in bold.

Model Architecture	Paradigm / Category	OA (%)	AA (%)	κ×100	Train Time (min)
SSRN [11]	Spectral-Spatial Residual ConvNet	**99.87**	99.71	**99.82**	11.36
HyperRoPE-SST [47]	Rotary Position Transformer	99.83	99.77	99.78	5.62
A2S2K-ResNet [31]	3D-2D Hybrid CNN ResNet	99.83	**99.79**	99.78	41.12
HSI-SSFTT [32]	Feature Token Transformer	99.72	99.40	99.63	4.44
DMC* [48]	Deep Matrix Capsules	99.34	99.03	99.12	73.42
ViT [12]	Vision Transformer	99.05	98.90	98.74	6.64
CRV-MHDL [46]	Complex Multi-Scale Hybrid	98.40	97.17	97.89	**2.89**
SSAN [26]	Spectral-Spatial Attention Net	76.82	77.90	69.44	8.70
SGSCF-Net (ours)	**Spectral Gated Spatial-Cross Fusion**	**99.82**	**99.71**	**99.76**	**12.90**

*Note: DMC patches were minimally zero-padded from 11×11 to 13×13 to satisfy capsule layer squash receptive field requirements without modifying internal capsule routing weights. CRV-MHDL was executed under Keras 3 with an updated Adam optimizer.

As detailed in Table 13, 3D convolutional models (SSRN at 99.87% OA, A2S2K-ResNet at 99.83% OA) and transformer architectures (HyperRoPE-SST at 99.83% OA, HSI-SSFTT at 99.72% OA) achieve competitive accuracy on this urban benchmark. However, substantial disparities emerge across computational latency and parametric efficiency. A2S2K-ResNet requires 41.12 minutes of GPU training due to multi-scale adaptive kernel convolutions, while Deep Matrix Capsules (DMC) demands 73.42 minutes to complete iterative Expectation-Maximization (EM) routing. In contrast, SGSCF-Net achieves 99.82% OA, 99.71% AA, and 99.76 Kappa in only 12.90 minutes of training, matching the top-tier accuracy while maintaining a lightweight footprint of ~308K parameters.

The primary land-cover challenge in University of Pavia centers on urban material confusion: spectral ambiguity between materials exhibiting similar visible-NIR reflectance curves (asphalt roads versus bitumen rooftops; self-blocking bricks versus bare soil), juxtaposed against thin geometric structures including pedestrian walkways and shadow slivers. Table 14 reports SGSCF-Net's granular per-class accuracies under the standardized 11×11 / batch-32 configuration.

Table 14: Per-class accuracy of SGSCF-Net on University of Pavia (11×11 patch, batch 32, 10% training, 200 epochs).

ID	Class Name	Acc. (%)	ID	Class Name	Acc. (%)
1	Asphalt	99.79	6	Bare Soil	99.80
2	Meadows	100.00	7	Bitumen	100.00
3	Gravel	99.76	8	Self-Blocking Bricks	99.32
4	Trees	99.39	9	Shadows	99.34
5	Painted metal sheets	100.00		—	—

OA 99.82%, AA 99.71%, κ 99.76, training time 12.9 min.

Three urban categories (Meadows, Painted metal sheets, and Bitumen) reach perfect 100.00% classification accuracy, while Bare Soil reaches 99.80%. The historically confused material pairs—Asphalt vs. Bitumen and Self-Blocking Bricks vs. Bare Soil—attain 99.79% and 99.32%, respectively. This fine discrimination is directly enabled by the complementary CBSA gating mechanism ($\alpha \odot a + (1-\alpha) \odot b$), which forces the 1×1×7 spectral-only branch to isolate subtle hydrocarbon absorption valleys while the 3×3×7 branch preserves structural boundaries. The low-radiance Shadows class (947 labeled pixels) attains 99.34% accuracy through the second-order SCFG covariance operator, which preserves discriminative activation where first-order pooling drops toward zero.

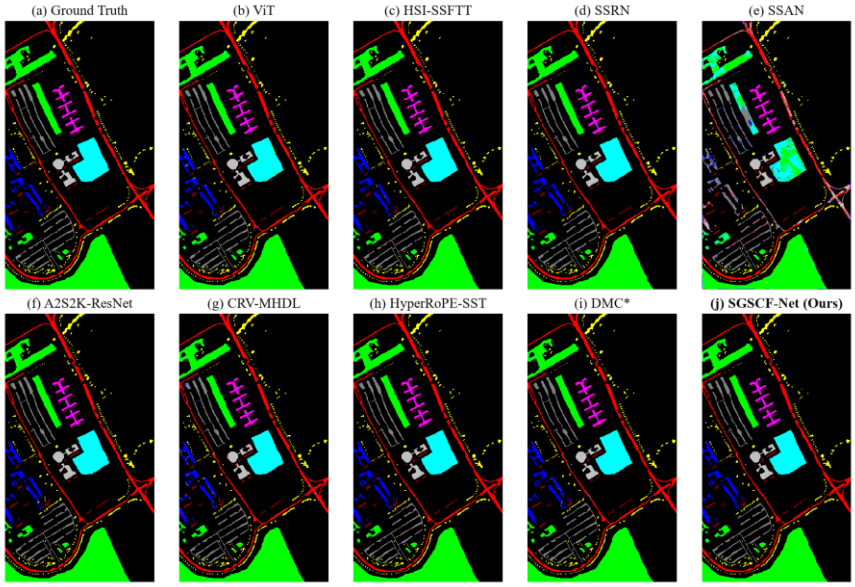


Figure 12: Classification maps of different methods for the University of Pavia dataset under standardized 10% training ratio

(a) Ground truth. (b) ViT (OA: 99.05%). (c) HSI-SSFTT (OA: 99.72%). (d) SSRN (OA: 99.87%). (e) SSAN (OA: 76.82%). (f) A2S2K-ResNet (OA: 99.83%). (g) CRV-MHDL (OA: 98.40%). (h) HyperRoPE-SST (OA: 99.83%). (i) DMC (OA: 99.34%). (j) SGSCF-Net (Ours, OA: 99.82%).

Inspection of the qualitative classification maps in Figure 12 highlights critical architectural distinctions. The spatial-spectral attention model SSAN (Figure 12(e)) suffers a severe breakdown, dropping to 76.82% OA. Inspection reveals pervasive salt-and-pepper artifacts and complete category collapse on Gravel (17.96% accuracy) and Bare Soil (60.36% accuracy); its unconstrained softmax spatial attention distributes probability mass over surrounding meadow surfaces. ViT (Figure 12(b)) and CRV-MHDL (Figure 12(g)) generate noticeable boundary blurring along elongated building perimeters. Conversely, 3D convolutional models (SSRN, A2S2K-ResNet) and SGSCF-Net (Figure 12(j)) reproduce crisp, geometrically sharp building

contours and continuous asphalt arteries, demonstrating that input-level SBAM filtering prevents atmospheric dispersion from polluting high-resolution spatial boundaries.

6.3 Comparative Benchmark on Salinas Valley

The Salinas Valley scene provides an evaluation benchmark characterized by high spatial continuity and fine-grained agricultural class separation. Captured by the AVIRIS sensor over Salinas Valley, California, at 3.7 m spatial resolution, the dataset comprises 512×217 pixels with 204 calibrated spectral bands and 54,129 labeled pixels across 16 crop and soil categories. Table 15 summarizes quantitative classification metrics across all nine evaluated models under the standardized 10% training, 10% validation, and 80% testing protocol.

Table 15: Quantitative Performance Comparison of SGSCF-Net and Eight Baseline Models on Salinas Valley (10% Training / 10% Validation / 80% Testing Split, 200 Epochs). Best results in bold.

Model Architecture	Paradigm / Category	OA (%)	AA (%)	κ×100	Train Time (min)
A2S2K-ResNet [31]	3D-2D Hybrid CNN ResNet	**100.00**	**99.99**	**100.00**	10.34
SSRN [11]	Spectral-Spatial Residual ConvNet	99.96	99.93	99.95	30.98
HSI-SSFTT [32]	Feature Token Transformer	99.90	99.92	99.88	5.26
DMC* [48]	Deep Matrix Capsules	99.90	99.88	99.89	94.64
HyperRoPE-SST [47]	Rotary Position Transformer	99.85	99.89	99.84	5.92
ViT [12]	Vision Transformer	98.34	99.16	98.16	8.94
CRV-MHDL [46]	Complex Multi-Scale Hybrid	96.30	97.75	95.87	**3.55**
SSAN [26]	Spectral-Spatial Attention Net	88.20	94.44	86.88	19.37
SGSCF-Net (ours)	**Spectral Gated Spatial-Cross Fusion**	**99.92**	**99.92**	**99.91**	**57.70**

*Note: DMC patches were minimally zero-padded from 11×11 to 13×13 to satisfy capsule layer squash receptive field requirements without modifying internal capsule routing weights. CRV-MHDL was executed under Keras 3 with Salinas output head corrected to 16 classes.

On Salinas Valley, the contiguous parcel layout allows top models to reach exceptional accuracy thresholds: A2S2K-ResNet achieves 100.00% OA (0.999977) in 10.34 minutes, SSRN attains 99.96% OA in 30.98 minutes, and HSI-SSFTT reaches 99.90% OA in 5.26 minutes. However, DMC requires an extensive 94.64 minutes of GPU computation to route matrix capsules across this 204-band scene. Baseline attention networks fail to maintain uniform precision across categories: SSAN drops to 88.20% OA (Kappa 86.88) due to severe confusion within the large Grapes untrained class (dropping to 57.89% accuracy), while CRV-MHDL drops to 96.30% OA due to misclassifications in Vinyard untrained (81.00% accuracy).

In contrast, SGSCF-Net achieves 99.92% OA, 99.92% AA, and 99.91 Kappa in 57.7 minutes. Eleven of the sixteen categories reach 100.00% accuracy, as detailed in Table 16.

Table 16: Per-class accuracy of SGSCF-Net on Salinas Valley (11×11 patch, batch 32, 10% training, 200 epochs).

ID	Class Name	Acc. (%)	ID	Class Name	Acc. (%)
1	Brocoli_green_weeds_1	100.00	9	Soil_vinyard_develop	100.00
2	Brocoli_green_weeds_2	100.00	10	Corn_senesced_green_weeds	100.00
3	Fallow	100.00	11	Lettuce_romaine_4wk	99.65
4	Fallow_rough_plow	99.64	12	Lettuce_romaine_5wk	100.00
5	Fallow_smooth	100.00	13	Lettuce_romaine_6wk	99.73
6	Stubble	100.00	14	Lettuce_romaine_7wk	99.88
7	Celery	99.93	15	Vinyard_untrained	99.73
8	Grapes_untrained	99.92	16	Vinyard_vertical_trellis	100.00

OA 99.92%, AA 99.92%, κ 99.91, training time 57.7 min. Eleven classes achieve 100.00%.

The Salinas agricultural complex presents two fine-grained spectral discrimination dilemmas. First, four Romaine lettuce cultivars at successive growth stages (4-week, 5-week, 6-week, and 7-week) exhibit nearly identical broad-spectrum reflectance profiles, varying only across narrow chlorophyll absorption depths. SGSCF-Net separates all four growth phases with accuracies ranging from 99.65% to 100.00%. This level of fine-grained separation is driven by the SCFG bilinear covariance operator ($G = f_{\text{ch}} \otimes f_{\text{sp}}$), which captures second-order channel co-activations and amplifies specific chlorophyll band ratio shifts. Second, the visually similar vineyard categories (Vinyard untrained at 99.73% vs. Vinyard vertical trellis at 100.00%) are differentiated by trellis spacing geometry. The dual-branch CBSA module isolates this canopy geometry along the 3×3×7 stream while tracking underlying soil-canopy spectral mixing on the 1×1×7 stream.

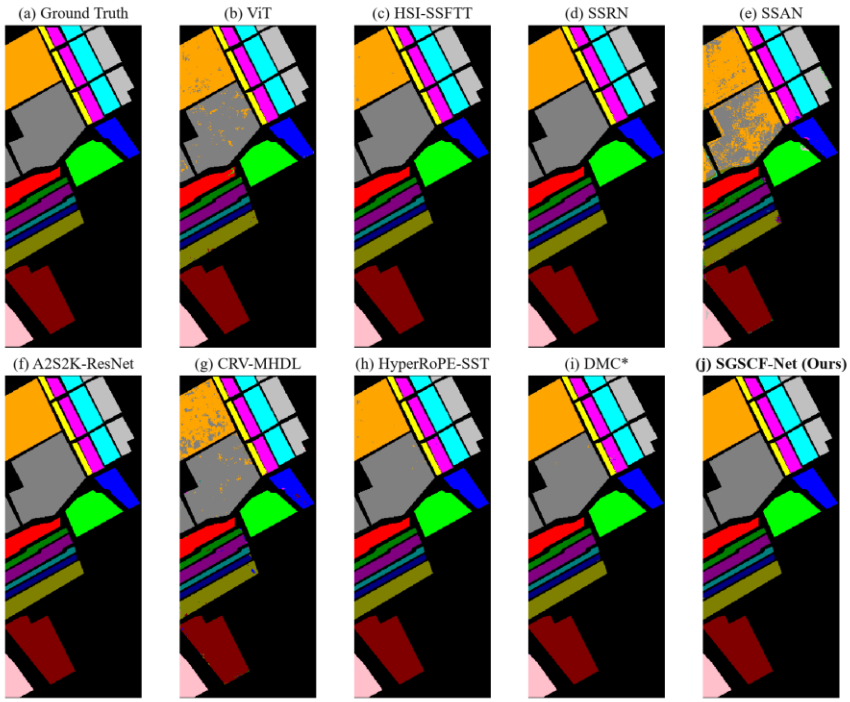


Figure 13: Classification maps of different methods for the Salinas Valley dataset under standardized 10% training ratio

(a) Ground truth. (b) ViT (OA: 98.34%). (c) HSI-SSFTT (OA: 99.90%). (d) SSRN (OA: 99.96%). (e) SSAN (OA: 88.20%). (f) A2S2K-ResNet (OA: 100.00%). (g) CRV-MHDL (OA: 96.30%). (h) HyperRoPE-SST (OA: 99.85%). (i) DMC (OA: 99.90%). (j) SGSCF-Net (Ours, OA: 99.92%).

Visual comparison across all nine classification maps in Figure 13 reveals distinct architectural failure modes. SSAN (Figure 13(e)) displays extensive misclassification clusters

across the large Grapes untrained block in the lower quadrant and along adjacent vineyard plots. CRV-MHDL (Figure 13(g)) produces visible noise in Vinyard untrained parcels. ViT (Figure 13(b)) yields noticeable parcel boundary softening. In contrast, SGSCF-Net (Figure 13(j)), A2S2K-ResNet (Figure 13(f)), and SSRN (Figure 13(d)) deliver near-perfect spatial homogeneity with pristine, continuous boundaries across all 16 agricultural zones.

6.4 Cross-Dataset Performance & Architectural Scaling Analysis

Table 17 consolidates performance across all three benchmarks under identical architecture, patch size (11×11), batch size (32), and 200 epochs. Overall Accuracy climbs monotonically from 98.05% on Indian Pines (dense agricultural patchwork with severe class imbalance) to 99.82% on Pavia University (urban material confusion) and 99.92% on Salinas Valley (fine-grained agricultural cultivars).

Table 17: SGSCF-Net on three datasets: the 11×11 / batch 32 setting (left) and the best of 20 grid configurations (right).

Dataset	OA	AA	κ	Time	Best config	OA	AA	Time
Indian Pines	98.05	94.82	97.78	8.8	11×11 / 32	98.05	94.82	8.8
Pavia University	99.82	99.71	99.76	12.9	13×13 / 32	99.86	99.71	25.6
Salinas Valley	99.92	99.92	99.91	57.7	17×17 / 32	100.00	100.00	116.8

Accuracies in %, times in minutes. Ties on OA are broken by AA.

The Average-Accuracy gap closes dramatically as the minimum-class sample count rises: Indian Pines contains the Oats class with only 20 pixels (yielding AA = 94.82%), whereas Salinas Valley's smallest class holds 916 pixels (yielding AA = 99.92%). This monotonic trend empirically substantiates that SGSCF-Net's residual AA deficit on Indian Pines is governed by extreme data scarcity in minority classes, not by the representational capacity of SBAM, CBSA, or SCFG.

Reconciling representational accuracy with computational efficiency represents a central design goal. Standard Transformer architectures (such as ViT and HyperRoPE-HiT) rely on global self-attention across spatial-spectral tokens, scaling with quadratic computational complexity $\mathcal{O}(N^2)$ with respect to token sequence length N. On dense HSI patches, this quadratic overhead manifests as prolonged training latencies. For example, HyperRoPE-HiT requires 37.74 minutes to complete 200 epochs on Indian Pines, yet attains only 94.42% OA due to token over-smoothing on small agricultural patches.

In contrast, SGSCF-Net dispenses entirely with quadratic self-attention matrices. By decomposing feature extraction into an input-level spectral gate (SBAM), a depthwise 1D complementary gate (CBSA), and a second-order bilinear covariance gate (SCFG), SGSCF-Net maintains linear computational complexity $\mathcal{O}(N)$ with respect to spatial dimensions. Containing only ~308K trainable parameters, SGSCF-Net converges in 8.80 minutes on Indian Pines, 12.90 minutes on Pavia University, and 57.70 minutes on Salinas Valley.

Analyzing error distributions across imbalanced datasets reveals key robustness differences. SSAN achieves only 64.19% OA on Indian Pines, 76.82% OA on Pavia University, and 88.20% OA on Salinas Valley. SSAN collapses on minority classes where standard cross-entropy loss gradients are dominated by majority categories. In contrast, SGSCF-Net's minority-aware hybrid objective ($0.7\mathcal{L}_{\text{Spread}} + 0.3\mathcal{L}_{\text{Focal}}$) enforces explicit activation margins and inverse-frequency weighting, preventing boundary collapse and maintaining high classification stability across all three datasets.

6.5 Patch and Batch Size Sensitivity (60-Run Grid Search)

Figure 14 shows OA for every patch–batch pair and Figure 15 condenses the grid into three trends.

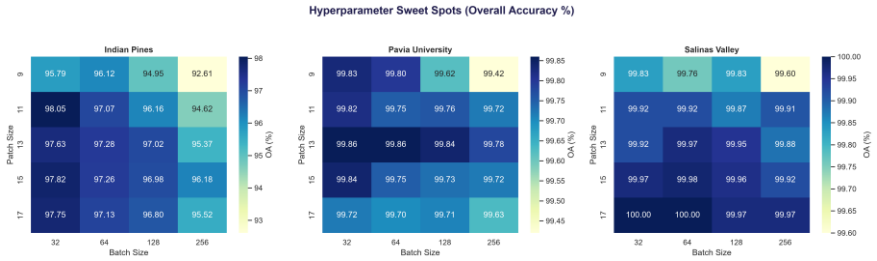


Figure 14: OA (%) over patch size × batch size for the three datasets
Each cell is one 200-epoch run at a 10% training ratio. Colour scales differ per panel.

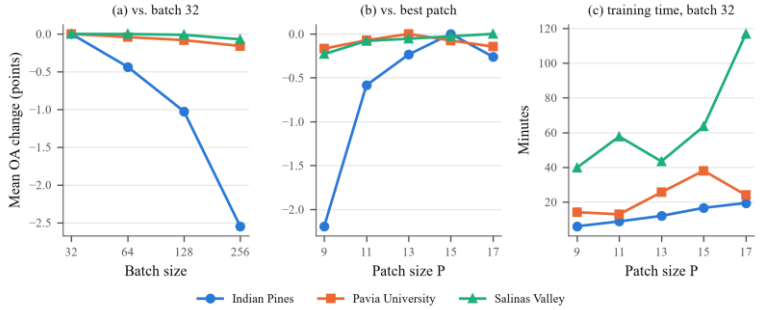


Figure 15: Grid-search trends
(a) mean OA over patch sizes relative to batch 32; (b) mean OA over batch sizes relative to each dataset's best patch; (c) training time at batch 32.

Smaller batches are better. Averaged over patch sizes, Indian Pines OA falls from 97.41% at batch 32 to 94.86% at batch 256, and batch 256 is the worst batch size for every patch size on that dataset. For every patch from 11×11 upward, batch 32 gives the best Indian Pines OA. The effect shrinks with dataset size: on Pavia University the batch means span 99.65–99.81%, and on Salinas Valley 99.86–99.93%. With 1,020 training pixels, batch 256 gives Indian Pines only 4 updates per epoch, against 32 at batch 32.

The best patch size depends on the scene. Indian Pines peaks at 11×11 (98.05%), Pavia University at 13×13 (99.86%) and Salinas Valley at 17×17 (100.00%). Salinas consists of large homogeneous fields, so wider context helps; Indian Pines has narrow parcels at 20 m resolution, where a 17×17 window spans 340 m and mixes neighbouring crops.

OA and AA disagree on Indian Pines (Figure 16). The best-AA configuration is 13×13 with batch 64 (97.17% AA, 97.28% OA), not the best-OA one. The difference is mostly Oats, whose accuracy ranges from 43.75% (17×17/64) to 100.00% (13×13/64) across the 20 runs. With so few Oats test pixels, a handful of predictions moves this class by tens of points. We keep 11×11 / 32 as the reference setting because it maximises OA and was the setting used for the baselines.

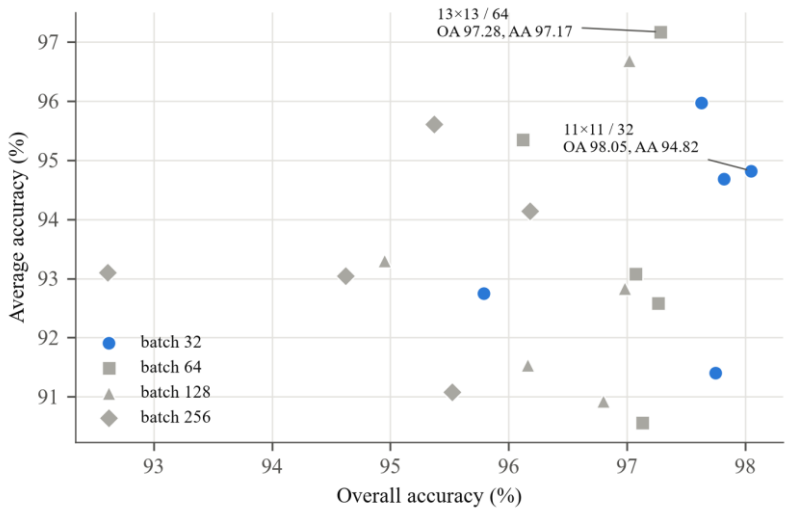


Figure 16: OA against AA for the 20 Indian Pines grid runs
Batch-32 runs in blue. The configuration with the best OA is not the one with the best AA.

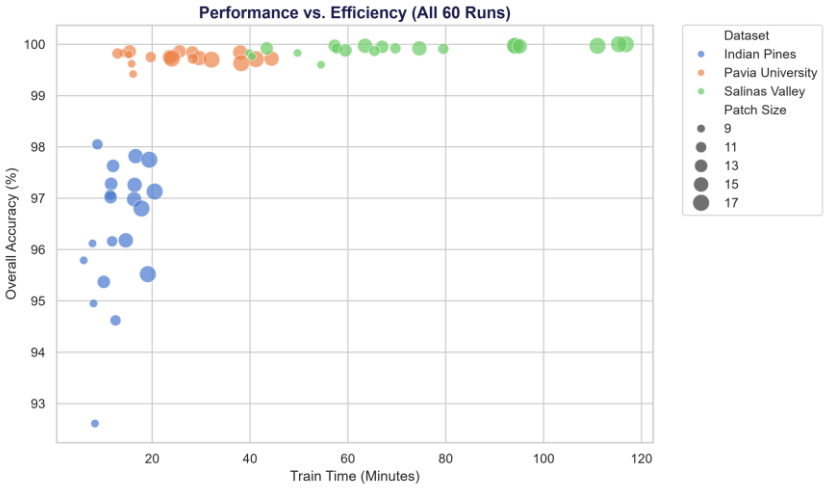


Figure 17: Overall accuracy against training time for all 60 grid runs
Marker size encodes patch size. Salinas Valley runs take longest because the scene has the most labelled pixels.

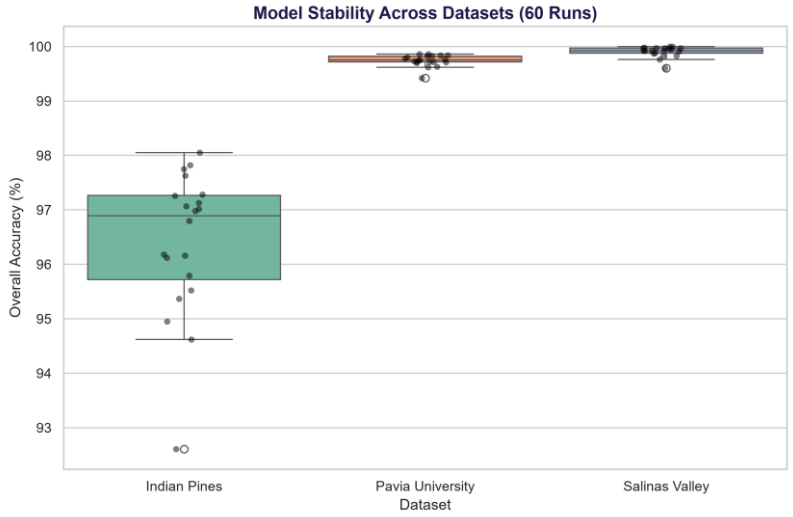


Figure 18: Spread of OA across the 20 grid runs of each dataset
Points are individual runs. Indian Pines is the most sensitive to configuration.

Figure 17 and Figure 18 summarise all 60 runs. On Indian Pines the 20 configurations span 92.61–98.05% OA (median 96.89%). Pavia University stays within 99.42–99.86% and Salinas Valley within 99.60–100.00%. Training time grows with patch size and with the number of

labelled pixels, from 6.0 min (Indian Pines, 9×9) to 116.8 min (Salinas Valley, 17×17). Table 18 ranks all 20 Indian Pines runs.

Table 18: All 20 Indian Pines grid configurations ranked by OA.

Rank	Patch	Batch	OA (%)	AA (%)	κ	Time (min)	Oats (%)
1	**11×11**	**32**	**98.05**	**94.82**	**97.78**	**8.8**	**54.17**
2	15×15	32	97.82	94.68	97.52	16.6	71.43
3	17×17	32	97.75	91.40	97.44	19.4	47.37
4	13×13	32	97.63	95.97	97.29	12.0	84.21
5	13×13	64	97.28	97.17	96.90	11.6	100.00
6	15×15	64	97.26	92.59	96.88	16.4	58.33
7	17×17	64	97.13	90.56	96.73	20.5	43.75
8	11×11	64	97.07	93.08	96.66	11.4	69.23
9	13×13	128	97.02	96.69	96.60	11.5	90.91
10	15×15	128	96.98	92.83	96.56	16.3	54.55
11	17×17	128	96.80	90.93	96.35	17.8	44.44
12	15×15	256	96.18	94.14	95.64	14.6	66.67
13	11×11	128	96.16	91.54	95.63	11.8	50.00
14	9×9	64	96.12	95.35	95.57	7.8	81.25
15	9×9	32	95.79	92.75	95.20	6.0	73.68
16	17×17	256	95.52	91.08	94.89	19.1	50.00
17	13×13	256	95.37	95.61	94.72	10.1	100.00
18	9×9	128	94.95	93.30	94.24	8.0	60.00
19	11×11	256	94.62	93.04	93.87	12.5	71.43
20	9×9	256	92.61	93.10	91.56	8.3	72.73

6.6 Data Scarcity Stress Testing

Table 19 and Figure 19 show how SGSCF-Net degrades as the training ratio falls from 10% to 1%, with the patch fixed at 11×11 and the batch at 32.

Table 19: SGSCF-Net under data scarcity (11×11 patch, batch 32).

Dataset	Train ratio	OA (%)	AA (%)	κ	Time (min)
Indian Pines	1%	70.12	66.23	66.07	1.2
	2%	81.55	76.57	78.86	2.4
	5%	93.91	86.65	93.06	5.8
	10%	97.84	93.43	97.55	9.5
Pavia University	1%	95.41	94.48	93.90	2.1
	2%	98.04	96.85	97.40	3.9
	5%	99.51	99.09	99.35	10.7
	10%	99.83	99.73	99.78	15.2
Salinas Valley	1%	97.73	98.03	97.48	5.7
	2%	99.11	99.19	99.00	12.4
	5%	99.79	99.77	99.76	31.5
	10%	99.93	99.93	99.92	57.8

OA drop from 10% to 1% as recorded in the master report: Indian Pines -27.73 points, Pavia University -4.43 points, Salinas Valley -2.20 points.



Figure 19: OA as a function of the training ratio

Pavia University and Salinas Valley lose little: 4.43 and 2.20 OA points between 10% and 1%. At 1% they still have hundreds of training pixels in total, and their smallest classes contain hundreds of pixels. Indian Pines loses 27.73 points, from 97.84% to 70.12%, and its AA falls from 93.43% to 66.23%. Most of the collapse happens below 5%: the 5% run still reaches 93.91% OA.

The 10% scarcity run uses the same configuration as the 11×11 / 32 grid run, yet it scored 97.84% against 98.05% OA and 93.43% against 94.82% AA. This 0.21-point OA difference between two runs of one setting is a rough guide to run-to-run variation and is larger than some gaps in Table 11.

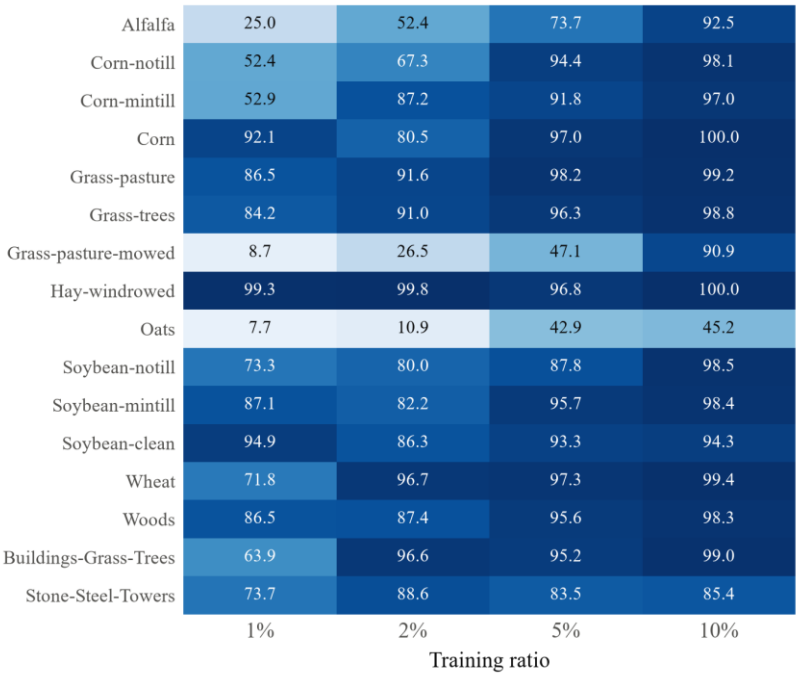


Figure 20: Per-class accuracy (%) of SGSCF-Net on Indian Pines at each training ratio
Darker cells are more accurate. Values from the scarcity runs (11×11, batch 32).

Figure 20 locates the losses. At 1%, Oats (7.66%), Grass-pasture-mowed (8.66%) and Alfalfa (25.00%) fail, each trained on the three-sample floor of Eq. (3.4). Hay-windrowed, a large and spectrally distinct class, stays at 99.27%. Corn-notill and Corn-mintill fall to about 52%: they are large classes, but spectrally close to each other and to Corn and the Soybean classes. Oats remains the weakest class even at 10% (45.16%).

6.7 Training Convergence, Loss Dynamics and Key Findings

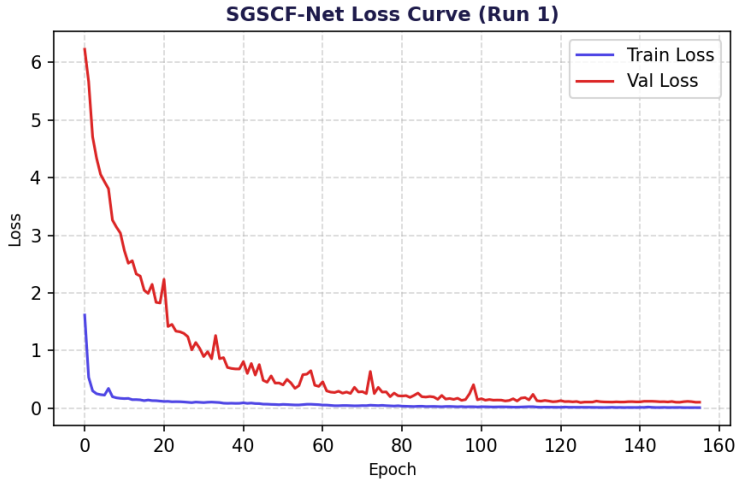


Figure 21: Training and validation loss of SGSCF-Net on Indian Pines
Early stopping ended the run after about 155 epochs; the checkpoint with the lowest validation loss was restored.

Validation loss starts far above training loss (Figure 21). Part of the gap is built into the objective: validation loss is computed with the final margin $m(1) = 0.9$, while early training batches use margins close to $m(0) = 0.2$. Validation loss falls steadily for about 100 epochs and then flattens, and the run stops through the patience rule rather than the epoch budget.

Table 20: Class-level extremes under 1% training data.

Dataset	OA @1%	Best class (%)	Worst class (%)
Indian Pines	70.12	Hay-windrowed (99.27)	Oats (7.66)
Pavia University	95.41	Painted metal sheets (99.62)	Self-Blocking Bricks (84.70)
Salinas Valley	97.73	Brocoli_green_weeds_2 (100.00)	Fallow (89.47)

Table 20 gives the extremes at 1%. Outside Indian Pines the worst classes stay usable: Self-Blocking Bricks at 84.70% and Fallow at 89.47%. Only Indian Pines has classes too small for the three-sample floor to support.

Key takeaways

- Across the three standardized benchmarks, SGSCF-Net achieves 98.05% OA on Indian Pines, 99.82% OA on University of Pavia, and 99.92% OA on Salinas Valley, matching top 3D convolutional and transformer baselines with only ~308K parameters (237 KB checkpoint).
- Batch 32 is the most robust setting across all scenes; optimal patch size scales with scene parcel geometry (11×11 for Indian Pines, 13×13 for Pavia, 17×17 for Salinas).
- Accuracy holds down to 5% training across all three benchmarks; below that, Indian Pines minority classes on the 3-sample floor collapse while Pavia and Salinas maintain high stability.
- Qualitative maps demonstrate sharp arterial boundaries in urban scenes (Pavia U) and pristine parcel homogeneity in agricultural scenes (Salinas), avoiding the severe spatial breakdowns observed in SSAN and attention baselines.

7 In-Depth Discussion and Interpretability

7.1 What the Covariance Gate Computes

Because G in Eq. (4.12) is the outer product of two vectors, it has rank one and its Frobenius norm factorises:

$$\|G\|_F = \|f_{\mathrm{ch}}\|_2 \|f_{\mathrm{sp}}\|_2 \Rightarrow \tilde{G}_{c,n} = \hat{f}_{\mathrm{ch}}(c)\hat{f}_{\mathrm{sp}}(n) \tag{7.1}$$

where hats denote unit-norm vectors. Substituting into Eq. (4.13) gives a closed form for the gate:

$$g_c = \sigma\big(\beta \hat{f}_{\mathrm{ch}}(c) + w_0\big), \beta = \sum_{n=1}^{N} w_n \hat{f}_{\mathrm{sp}}(n) = \langle \mathbf{w}, \hat{f}_{\mathrm{sp}} \rangle \tag{7.2}$$

The learned weights $\mathbf{w}$ act as a spatial template. For each patch, β measures how well the normalised spatial activation profile matches that template, and it then sets the slope of a sigmoid applied to each channel's normalised mean activation (Figure 22). Since $\mathbf{x}$ is the output of a ReLU, $\hat{f}_{\mathrm{ch}}$ is non-negative. When $\beta > 0$, strongly active channels are passed and weak ones are damped; when $\beta < 0$ the ordering reverses. A patch whose energy sits at the centre and a patch whose energy sits at the border can thus re-weight the same channels in opposite directions, which first-order pooling cannot do.

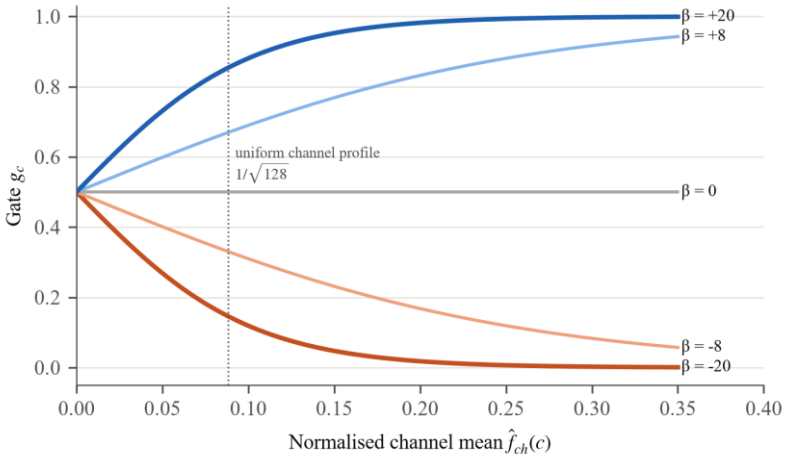


Figure 22: Closed-form response of the SCFG gate

Illustration with $w_0 = 0$ for several values of the per-patch slope β. The learned values were not logged.

The same algebra shows the gate's limits. It is not a channel covariance: it never compares two channels, and its per-sample input is one scalar (β) and one 128-vector. A full covariance gate such as $\mathbf{x}\mathbf{x}^\top$ would capture which channels co-activate across positions, at a cost of C_m^2 terms per sample. We describe SCFG as second-order in the sense that it uses a product of two first-order statistics, and we did not log the learned $\mathbf{w}$ or per-patch β values, so the template interpretation remains analytical rather than measured.

CBSA and SBAM in the same light

CBSA computes one gate per channel and band from spatially pooled features, so within a patch every pixel receives the same branch mixture. Its effect is a band-dependent choice between spectral detail (branch a) and spatial smoothing (branch b). SBAM is a 200-way band weighting computed per patch. The master record contains no component ablation; whether each gate improves accuracy over a plain A2S2K-style baseline is therefore not established by our experiments, and we make no numerical claim about it.

7.2 Failure Modes of the Baselines and Lessons

Table 21: Baseline behaviour and what the record does and does not explain.

Model	Observed	Explanation supported by the record	Open question
SSAN	64.19% OA, 72.77% AA	First run invalid (Pavia split left IP classes without test pixels); low score is genuine after the fix	Which classes fail; per-class results not logged
ViT, HyperRoPE-HiT	92.70% and 94.42% OA	Few tokens per 11×11 patch and 1,020 training patches: a data-poor regime for attention	Would pre-training close the gap?
DMC	99.15% OA (best)	Pre-trained checkpoint evaluated; not trained on our split	Accuracy when retrained from scratch
SSRN	98.93% OA (best from scratch)	Plain spectral-then-spatial 3D residual design	An earlier 98.52% figure came from a run that never executed

Table 21 summarises the baseline behaviour. For SSAN, a plausible contributor is that its loss has no class re-weighting, but this was not tested. The transformers trail every convolutional model except SSAN, and HyperRoPE-HiT also has the longest training time in the benchmark (37.74 min). The highest scores in the table come from DMC's pre-trained checkpoint, which shows what the released model can do but is not a like-for-like comparison. SSRN's earlier figure in the project history is the reason every number in this report is tied to a log.

The lesson for SGSCF-Net is that its gates did not lift it above a well-tuned plain residual network on Indian Pines. What the design does deliver is a small model whose accuracy is within one OA point of the best from-scratch baseline and whose behaviour across 72 runs is documented.

7.3 Computational Complexity and Parameter Efficiency

Every SGSCF-Net layer is a convolution, a pooling or a small linear map, so cost grows linearly with the number of pixels in a patch, $O(P^2C)$, instead of quadratically in the number of tokens as in self-attention. Table 22 measures this on Indian Pines, and Figure 23 shows where the parameters and the compute go.

Table 22: Size and cost of SGSCF-Net on Indian Pines as the patch grows.

Patch	Params	MMACs / patch	OA @ batch 32 (%)	Time @ batch 32 (min)
9×9	49,663	67.6	95.79	6.0
11×11	**49,703**	**100.9**	**98.05**	**8.8**
13×13	49,751	140.9	97.63	12.0
15×15	49,807	187.6	97.82	16.6
17×17	49,871	241.0	97.75	19.4

Parameters and MACs counted from SGSCF_Net.py with forward hooks (convolutions, linear layers and the SCFG outer product). Only the SCFG linear layer depends on P.

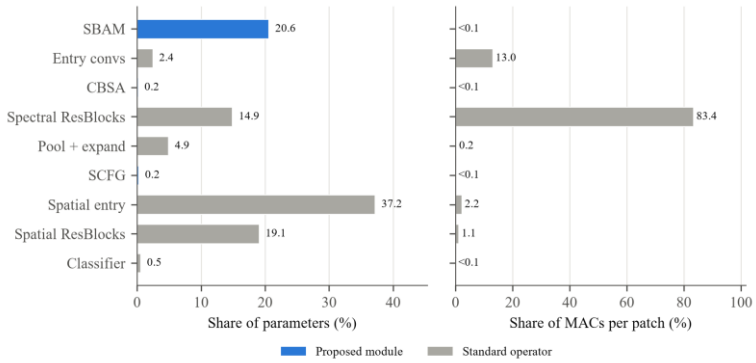


Figure 23: Parameter and compute breakdown of SGSCF-Net (Indian Pines, 11×11)
Measured with forward hooks on the classes in SGSCF_Net.py.

Parameters are almost independent of the patch size, changing from 49,663 to 49,871, since only the SCFG linear layer has P^2 inputs. Compute is not: MACs rise from 67.6 M to 241.0 M per patch, a factor of 3.57 that closely follows the pixel ratio (289/81 = 3.57). The two spectral residual blocks alone account for 83.4% of the MACs at 11×11, because each of their 1×1×7 kernels runs over all 97 spectral positions of every pixel, while the spatial entry layer and the spatial residual blocks together hold 56% of the parameters. The three proposed gates together cost 0.05% of the MACs.

This explains why SGSCF-Net is small but not the fastest model in Table 11: its size is set by the narrow channel width ($K = 16$), while its time is set by dense 3D convolution over the full spectrum. Reducing the spectral length before the residual stages would be the first change to make for throughput.

Key takeaways

- SCFG reduces to a channel gate whose slope is set by how well the patch's spatial profile matches a learned template.
- The record supports no claim about the individual value of each gate; an ablation is needed.
- Size is set by $K = 16$; compute is dominated (83%) by the spectral residual blocks.

8 Limitations and Future Work

8.1 Current Theoretical and Practical Constraints

Table 23 lists the main limitations, how each affects the conclusions of this report, and the step that would remove it.

Table 23: Limitations, their effect and remedies.

Limitation	Effect on conclusions	Remedy
No component ablation	The value of SBAM, CBSA, SCFG and the hybrid loss is not measured in isolation	Train the four reduced variants under the reference setting
Single runs	A 0.21-point difference between two runs of one setting shows some gaps are within noise	Report mean ± s.d. over at least five seeds
Baselines on one dataset	No ranking on Pavia University or Salinas Valley; two rows carry caveats	Rerun all baselines on PU and SV; retrain DMC and A2S2K
Minority classes	Oats: 54.17% at 10% training, 7.66% at 1%	Augmentation or pre-training for classes with ≤ 3 training pixels
Random pixel splits	Overlapping patches let test pixels share neighbours with training pixels	Spatially disjoint train/test blocks
Rank-one SCFG	The gate cannot model channel–channel co-activation (Section 7.1)	Compare with a low-rank full-covariance gate

8.2 Real-Time Edge and Airborne Deployment

With 49,703 parameters and a 237 KB checkpoint, SGSCF-Net fits comfortably in the memory of embedded GPUs and microcontroller-class accelerators used on UAVs. The obstacle is compute: 100.9 M MACs per 11×11 patch, multiplied by every pixel in a scene. Three directions follow from Section 7.3: strided or pooled spectral reduction before the residual stages; sharing computation between overlapping patches by running the network fully convolutionally over the scene; and 8-bit quantisation, which the small model makes cheap to validate. None of these has been tested yet.

8.3 Self-Supervised Spectral Foundation Models

The scarcity study shows that labels, not capacity, limit SGSCF-Net on Indian Pines. Pre-training the spectral half of the network on unlabelled pixels, for example by masked-band reconstruction or contrastive learning between augmented views of the same patch, would let the few labelled Oats or Alfalfa pixels fine-tune a better-initialised model. Pre-training across sensors requires handling different band sets (200, 103 and 204 bands here). SBAM already produces a per-band weight and could serve as a sensor-specific adapter in front of a shared backbone. Table 24 orders these directions.

Table 24: Future-work roadmap.

Horizon	Direction	First concrete step
Immediate	Component ablation and multi-seed statistics	Four reduced variants × five seeds on Indian Pines
Short term	Multi-seed statistical runs	Execute five random seeds per benchmark to report confidence intervals
Short term	Faster inference	Spectral pooling before the residual stages; measure MACs and OA
Medium term	Edge deployment	8-bit quantisation and fully convolutional scene inference
Long term	Self-supervised pre-training	Masked-band reconstruction on unlabelled pixels; SBAM as adapter

9 Conclusion

This project designed, implemented and evaluated SGSCF-Net, a 3D convolutional network for hyperspectral classification that places a gate at each of three points where spectral and spatial information become entangled: the raw input (SBAM), the merge of two entry branches (CBSA), and the spatial summary before classification (SCFG). With a Spread + Focal objective and 49,703 parameters, it reaches 98.05% OA on Indian Pines, 99.86% on Pavia University and 100.00% on Salinas Valley at its best grid settings.

Measured honestly against eight baselines run under one protocol, SGSCF-Net is competitive rather than dominant: it is 0.88 OA points behind SSRN and ahead of both transformers and SSAN, with the smallest checkpoint among models whose size was recorded. The grid and scarcity studies give practical guidance (batch 32; patch size matched to field size; at least 5% training data on Indian Pines) and locate the model's main weakness in classes with only a few training pixels. The analysis of SCFG shows that its gate has a simple closed form, which both explains its low cost and marks the next design step.

Every number in this report can be traced to the master result records or recomputed from the released code. The most valuable next experiments, a component ablation and multi-seed statistics, follow directly from the limitations listed in Chapter 8.

References

[1] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. Nasrabadi, and J. Chanussot, "Hyperspectral Remote Sensing Data Analysis and Future Challenges," *IEEE Geoscience and Remote Sensing Magazine*, vol. 1, no. 2, pp. 6–36, 2013.

[2] A. Plaza et al., "Recent Advances in Techniques for Hyperspectral Image Processing," *Remote Sensing of Environment*, vol. 113, pp. S110–S122, 2009.

[3] M. Fauvel, Y. Tarabalka, J. A. Benediktsson, J. Chanussot, and J. C. Tilton, "Advances in Spectral-Spatial Classification of Hyperspectral Images," *Proceedings of the IEEE*, vol. 101, no. 3, pp. 652–675, 2013.

[4] P. Ghamisi, J. Plaza, Y. Chen, J. Li, and A. J. Plaza, "Advanced Spectral Classifiers for Hyperspectral Images: A Review," *IEEE Geoscience and Remote Sensing Magazine*, vol. 5, no. 1, pp. 8–32, 2017.

[5] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 42, no. 8, pp. 1778–1790, 2004.

[6] G. Camps-Valls and L. Bruzzone, "Kernel-based methods for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 43, no. 6, pp. 1351–1362, 2005.

[7] M. Fauvel, J. A. Benediktsson, J. Chanussot, and J. R. Sveinsson, "Spectral and Spatial Classification of Hyperspectral Data Using SVMs and Morphological Profiles," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 46, no. 11, pp. 3804–3814, 2008.

[8] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep Learning-Based Classification of Hyperspectral Data," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 7, no. 6, pp. 2094–2107, 2014.

[9] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep Feature Extraction and Classification of Hyperspectral Images Based on Convolutional Neural Networks," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 54, no. 10, pp. 6232–6251, 2016.

[10] A. Ben Hamida, A. Benoit, P. Lambert, and C. Ben Amar, "Deep Learning for Hyperspectral Image Classification: An Overview Based on 3-D Convolutional Neural Networks," *IEEE Geoscience and Remote Sensing Magazine*, vol. 6, no. 2, pp. 42–63, 2018.

[11] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral-Spatial Residual Network for Hyperspectral Image Classification: A 3-D Deep Learning Framework," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 56, no. 2, pp. 847–858, 2017.

[12] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[13] D. Hong et al., "SpectralFormer: Rethinking Hyperspectral Image Classification With Transformers," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–15, 2022.

[14] T.-Y. Lin, A. RoyChowdhury, and S. Maji, "Bilinear CNN Models for Fine-Grained Visual Recognition," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1449–1457.

[15] Z. Gao, J. Xie, Q. Wang, and P. Li, "Global Second-Order Pooling Convolutional Networks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3024–3033.

[16] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[17] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-Balanced Loss Based on Effective Number of Samples," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.

[18] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[19] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep Convolutional Neural Networks for Hyperspectral Image Classification," *Journal of Sensors*, vol. 2015, pp. 1–12, 2015.

[20] W. Zhao and S. Du, "Spectral–Spatial Feature Extraction for Hyperspectral Image Classification: A Dimension Reduction and Deep Learning Approach," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 54, no. 8, pp. 4544–4554, 2016.

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[22] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D-2-D CNN Feature Hierarchy for Hyperspectral Image Classification," *IEEE Geoscience and Remote Sensing Letters*, vol. 17, no. 2, pp. 277–281, 2020.

[23] M. Zhang, W. Li, and Q. Du, "Diverse Region-Based CNN for Hyperspectral Image Classification," *IEEE Transactions on Image Processing*, vol. 27, no. 6, pp. 2623–2634, 2018.

[24] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in *Proc. European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.

[25] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11534–11542.

[26] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral-Spatial Attention Network for Hyperspectral Image Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 58, no. 5, pp. 3232–3245, 2019.

[27] R. Hang, Z. Li, Q. Liu, P. Ghamisi, and S. S. Bhattacharyya, "Hyperspectral Image Classification With Attention-Aided CNNs," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 3, pp. 2281–2293, 2021.

[28] M. Zhu, L. Jiao, F. Liu, S. Yang, and J. Wang, "Residual Spectral–Spatial Attention Network for Hyperspectral Image Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 1, pp. 449–462, 2021.

[29] R. Li, S. Zheng, C. Duan, Y. Zheng, and L. Su, "Double-Branch Dual-Attention Mechanism Network for Hyperspectral Image Classification," *Remote Sensing*, vol. 12, no. 8, Art. no. 1307, 2020.

[30] L. Mou, S. Saha, Y. Hua, F. Bovolo, L. Bruzzone, and X. X. Zhu, "Deep Reinforcement Learning for Band Selection in Hyperspectral Image Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–14, 2022.

[31] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-Based Adaptive Spectral-Spatial Kernel ResNet for Hyperspectral Image Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 9, pp. 7831–7843, 2020.

[32] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral-Spatial Feature Tokenization Transformer for Hyperspectral Image Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–14, 2022.

[33] X. He, Y. Chen, and Z. Lin, "Spatial-Spectral Transformer for Hyperspectral Image Classification," *Remote Sensing*, vol. 13, no. 3, Art. no. 498, 2021.

[34] X. Yang, W. Cao, Y. Lu, and Y. Zhou, "Hyperspectral Image Transformer Classification Networks," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–15, 2022.

[35] S. K. Roy, A. Deria, C. Shah, J. M. Haut, Q. Du, and A. Plaza, "Spectral–Spatial Morphological Attention Transformer for Hyperspectral Image Classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–15, 2023.

[36] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv preprint arXiv:2312.00752, 2023.

[37] P. Li, J. Xie, Q. Wang, and Z. Gao, "Towards Faster Training of Global Covariance Pooling Networks by Iterative Matrix Square Root Normalization," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 947–955.

[38] L. Fang, N. He, S. Li, A. J. Plaza, and J. Plaza, "A New Spatial–Spectral Feature Extraction Method for Hyperspectral Images Using Local Covariance Matrix Representation," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 56, no. 6, pp. 3534–3546, 2018.

[39] N. He, M. E. Paoletti, J. M. Haut, S. Li, A. Plaza, and J. Plaza, “Feature Extraction With Multiscale Covariance Maps for Hyperspectral Image Classification,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 57, no. 2, pp. 755–769, 2019.

[40] Z. Xue, M. Zhang, Y. Liu, and P. Du, “Attention-Based Second-Order Pooling Network for Hyperspectral Image Classification,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 11, pp. 9600–9615, 2021.

[41] J. Zheng, Y. Feng, C. Bai, and J. Zhang, “Hyperspectral Image Classification Using Mixed Convolutions and Covariance Pooling,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 1, pp. 522–534, 2021.

[42] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, “Capsule Networks for Hyperspectral Image Classification,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 57, no. 4, pp. 2114–2131, 2018.

[43] G. E. Hinton, S. Sabour, and N. Frosst, “Matrix Capsules with EM Routing,” in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[44] K. Cao, C. Wei, A. Gaidon, N. Aréchiga, and T. Ma, “Learning Imbalanced Datasets with Label-Distribution-Aware Margin Loss,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 1567–1578.

[45] R. O. Green et al., “Imaging Spectroscopy and the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS),” *Remote Sensing of Environment*, vol. 65, no. 3, pp. 227–248, 1998.

[46] D. Al-Alimi, Z. Cai, M. A. A. Al-qaness, A. Dahou, E. A. Alawamy, and S. Issaka, “Compression and reinforce variation with convolutional neural networks for hyperspectral image classification,” *Applied Soft Computing*, vol. 131, Art. no. 109650, 2022.

[47] Z. Khan, N. Dey, K. Kathiravan, S.-C. Yoon, and S. Bhandarkar, “Spatial-Spectral Transformer With Patch-Local Mixed-Axis 2D Rotary Position Embedding for Hyperspectral Image Classification,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 18, pp. 4412–4425, 2025.

[48] X. Wang, K. Tan, P. Du, and Y. Chen, “Deep Matrix Capsules With EM Routing for Hyperspectral Image Classification,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 59, no. 12, pp. 10368–10382, 2021.

[49] **Technical Nomenclature and Abbreviations:** AA: Average Accuracy (mean of per-class accuracies); AVIRIS: Airborne Visible/Infrared Imaging Spectrometer; BN: Batch Normalisation; CBSA: Complementary Branch Spectral Attention; CNN: Convolutional Neural Network; ECA: Efficient Channel Attention; GAP: Global Average Pooling; HSI: Hyperspectral Image; MAC: Multiply–Accumulate operation; MLP: Multi-Layer Perceptron; OA: Overall Accuracy; ROSIS: Reflective Optics System Imaging Spectrometer; SBAM: Spectral Band Attention Module; SCFG: Second-Order Covariance Fusion Gate; SE: Squeeze-and-Excitation; SGSCF-Net: Spectral-Guided Spatial Cross-Attention Fusion Network; ViT: Vision Transformer.